

Prof. Dr. Frank José Affonso

Tutorial4Dev: Um Guia para Desenvolvedor Java

Rio Claro – SP

Agosto de 2023

Lista de figuras

Figura 1 – Tela para criação do projeto	7
Figura 2 – Tela para configuração dos testes	12
Figura 3 – Estrutura do projeto	13
Figura 4 – Resultado da execução	17
Figura 5 – Projeto e execução dos testes	20
Figura 6 – Diagrama de Classes UML	26
Figura 7 – Diagrama de Classes UML	27
Figura 8 – Estrutura do projeto	29
Figura 9 – Estrutura do Esquema	43
Figura 10 – Execução do teste	49
Figura 11 – Chamada da API via Postman: método getAllFisica	54
Figura 12 – Chamada da API via Postman: método getFisicaByCpf	56
Figura 13 – Chamada da API via Postman: método delete	57
Figura 14 – Chamada da API via Postman: método save - Configuração	62
Figura 15 – Chamada da API via Postman: método save - Resultado	63
Figura 16 – Chamada da API via Postman: método update	67
Figura 17 – Chamada da API via open-docs	70
Figura 18 – Chamada da API via Swagger	71
Figura 19 – Consulta de uma entidade pelo CPF via Swagger	72
Figura 20 – Documentando a API	74
Figura 21 – Códigos HTTP	77
Figura 22 – Comparativo antes e após a documentação do endpoint	79

Sumário

Introdução	4
I Linguagem de programação Java e o <i>framework</i> JUnit	5
1 Desenvolvendo testes unitários com o <i>framework</i> JUnit 5	6
1.1 O que é o JUnit 5?	6
1.2 Criando o projeto	7
1.3 Codificando nosso exemplo	11
1.4 Executando os testes	16
1.5 Suíte de teste	17
II Aplicações Java com o <i>framework</i> Spring Data	22
2 Desenvolvendo aplicações Java com Spring Data JPA	23
2.1 Falando um pouco do Spring Framework	23
2.2 Apresentando o exemplo	24
2.3 Criando o projeto	26
2.4 Implementação do exemplo	28
2.4.1 Preparação do projeto	28
2.4.2 Implementando a camada entity	29
2.4.3 Implementando a camada repository	34
2.4.4 Implementando a camada service	42
2.5 Escrevendo os casos de teste	45
2.6 Considerações finais	50
III APIs RESTful com o <i>framework</i> Spring	51
3 Desenvolvendo APIs RESTful com Spring Web	52
3.1 Considerações iniciais	52
3.2 Implementando a camada resource	52
3.3 Implementando a camada DTO	57
3.4 Implementando a camada resource (continuação...)	61
3.5 Considerações finais	68
4 Utilizando OpenAPI 3 e Swagger para documentação de APIs REST	69
4.1 Considerações Iniciais	69
4.2 Visão geral	69
4.3 Configurando o springdoc-openapi	70

4.4 Documentando nosso modelo	74
4.5 Considerações finais	79
Referências	80

Introdução

Este documento é formado por um conjunto de tutoriais que visa apoiar o projetista e/ou desenvolvedor Java na atividade de implementação de software. Como forma de apresentar os assuntos, este documento foi organizado em quatro partes:

- (i) **PARTE I.** O *framework* JUnit.
- (ii) **PARTE II.** Aplicações Java com Banco de Dados utilizando o *framework* Spring Data.
- (iii) **PARTE III.** APIs (do inglês, *Application Programming Interface*) RESTful com o *framework* Spring.

Parte I

Linguagem de programação Java e o
framework JUnit

1 Desenvolvendo testes unitários com o *framework* JUnit 5

Autor(es):

Prof. Dr. Frank J. Affonso

Pré-requisitos:

Java: Oracle JDK 64-Bit 17.0.4 LTS

JUnit 5: versão 5.9.0

JUnit 4: versão 4.13.2

Netbeans IDE: versão 14

O objetivo principal desta seção é mostrar aos interessados como o *framework* JUnit 5 pode ser utilizado na elaboração de testes unitários. Além disso, este documento visa apresentar também como a IDE¹ Netbeans pode facilitar essa atividade.

IMPORTANTE: O teste de software é uma atividade que está presente em diversos modelos prescritivos de desenvolvimento de software ([PRESSMAN, 2011](#); [SOMMERVILLE, 2011](#)). Essa atividade é abordada/adotada de diferentes maneiras nesses modelos, sendo utilizadas nas fases iniciais, intermediárias e finais da cadeia/ciclos de desenvolvimento. Vale destacar que o conteúdo apresentado neste documento não está associados aos modelos existentes na literatura, sendo apenas uma referência de utilização.

1.1 O que é o JUnit 5?

O JUnit é um *framework* para elaboração de testes unitários em Java, assim como outras linguagem compatíveis com a JVM (do inglês, *Java Virtual Machine*). Esse *framework* permite a criação de testes de maneira rápida e objetiva (ou seja, escrita e execução), visando ser uma alternativa viável para garantia da qualidade do código ([GAMMA et al., 1995](#); [BECHTOLD et al., 2007](#)). O JUnit 5 é composto por módulos de três subprojetos diferentes, os quais são detalhados a

¹ *Integrated Development Environment*

seguir (JUNIT, 2022):

JUnit 5 = Plataforma JUnit + JUnit Jupiter + JUnit Vintage

Plataforma JUnit. Serve como base para o lançamento de estruturas de teste na JVM, além de definir a API² `TestEngine` para desenvolver uma estrutura de teste executada na plataforma. Além disso, essa plataforma fornece um `Console Launcher` para iniciá-la a partir da linha de comando e um `Runner` baseado no JUnit 4 para executar qualquer `TestEngine` na plataforma em um ambiente baseado no JUnit 4.

JUnit Jupiter. É a combinação entre o novo modelo de programação e modelo de extensão para escrever testes e extensões no JUnit 5. O subprojeto `Jupiter` fornece um `TestEngine` para executar testes baseados na plataforma `Jupiter`.

JUnit Vintage. Fornece um `TestEngine` para executar testes baseados no JUnit 3 e JUnit 4 na plataforma.

1.2 Criando o projeto

Vamos trabalhar neste tutorial com projeto Maven da IDE Netbeans. Para isso, vá ao meu `File -> New Project`. Na janela que se abre, selecione a categoria `Java with Maven` e, em seguida, `Java Application`. Preencha os campos `Project Name` e `Group Id` conforme a Figura 1 e clique no botão `Finish`.

Figura 1 – Tela para criação do projeto

A imagem mostra a interface de criação de um novo projeto no NetBeans. A janela é dividida em duas partes principais. À esquerda, há uma lista de etapas: 1. Choose Project e 2. Name and Location. A etapa 2 está selecionada. À direita, sob o título 'Name and Location', há vários campos de texto e botões. Os campos preenchidos são: Project Name: JUnit5Tutorial; Project Location: ik/workspace/NetBeansProjects/2021/pos-graduacao (com um botão 'Browse...' ao lado); Project Folder: NetBeansProjects/2021/pos-graduacao/JUnit5Tutorial; Artifact Id: JUnit5Tutorial; Group Id: br.unesp.rc; Version: 1.0-SNAPSHOT; e Package: br.unesp.rc.junit5tutorial (com o rótulo '(Optional)' ao lado). Na base da janela, há uma barra de navegação com os botões '< Back', 'Next >', 'Finish' (destacado em azul), 'Cancel' e 'Help'.

Fonte: Elaborada pelo autor

Agora observe conteúdo do arquivo `pom.xml` do projeto na Listagem 1. Veja que o arquivo está “limpo” e não possui as configurações mínimas necessárias para trabalharmos com

² *Application Programming Interface*

testes. De acordo com orientação de configuração do manual do *framework*³ (BECHTOLD *et al.*, 2007), é necessário adicionar pelo menos uma implementação do TestEngine ao projeto para que o Maven Surefire ou o Maven Failsafe execute os testes.

Listagem 1 – Arquivo de configuração do projeto

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <groupId>br.unesp.rc.junit5tutorial</groupId>
8     <artifactId>JUnit5Tutorial</artifactId>
9     <version>1.0-SNAPSHOT</version>
10    <packaging>jar</packaging>
11    <properties>
12        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
13        <maven.compiler.source>17</maven.compiler.source>
14        <maven.compiler.target>17</maven.compiler.target>
15        <exec.mainClass>br.unesp.rc.junit5tutorial.JUnit5Tutorial</exec.mainClass>
16    </properties>
17 </project>
```

Fonte: Elaborada pelo autor

Antes iniciar a configuração, exclua a classe `JUnit5Tutorial` gerada no pacote do projeto e remova a Linha 17 apresentado na Listagem 1. Iniciando a configuração, adicione o trecho da Listagem 2 para configurar o suporte para testes baseados em JUnit Jupiter, além das dependências necessárias JUnit Jupiter API e JUnit Jupiter TestEngine.

Listagem 2 – Configuração da TestEngine

```
1 <build>
2     <plugins>
3         <plugin>
4             <artifactId>maven-surefire-plugin</artifactId>
5             <version>2.22.2</version>
6         </plugin>
7         <plugin>
8             <artifactId>maven-failsafe-plugin</artifactId>
9             <version>2.22.2</version>
10        </plugin>
11    </plugins>
12 </build>
13
```

³ <<https://junit.org/junit5/docs/current/user-guide/#running-tests-build-maven-engines-configure>>

```

14 <!-- ... -->
15
16 <dependencies>
17     <!-- ... -->
18     <dependency>
19         <groupId>org.junit.jupiter</groupId>
20         <artifactId>junit-jupiter-api</artifactId>
21         <version>5.9.0</version>
22         <scope>test</scope>
23     </dependency>
24     <dependency>
25         <groupId>org.junit.jupiter</groupId>
26         <artifactId>junit-jupiter-engine</artifactId>
27         <version>5.9.0</version>
28         <scope>test</scope>
29     </dependency>
30     <!-- ... -->
31 </dependencies>
32 <!-- ... -->

```

Fonte: Elaborada pelo autor

As dependências supracitadas podem ser realizadas de duas maneiras:

1. Pela IDE, clicando com o botão direito sobre a pasta *Dependencies*; ou
2. Pelo site *Maven repository*⁴.

Os passos dessas operações não serão apresentados neste tutorial. Na **Listagem 3** é apresentado o arquivo `pom.xml` do projeto. Observe na Linha 16 a criação da tag `<junit.jupiter.version>` para referenciar a versão do *framework* e os trechos entre as linhas 19-30 e 32-51 com as configurações e dependências recomendadas.

Listagem 3 – Configuração da TestEngine

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <groupId>br.unesp.rc</groupId>
8     <artifactId>JUnit5Tutorial</artifactId>
9     <version>1.0-SNAPSHOT</version>
10    <packaging>jar</packaging>

```

⁴ <<https://mvnrepository.com>>

```
11
12     <properties>
13         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14         <maven.compiler.source>17</maven.compiler.source>
15         <maven.compiler.target>17</maven.compiler.target>
16         <junit.jupiter.version>5.9.0</junit.jupiter.version>
17     </properties>
18
19     <build>
20         <plugins>
21             <plugin>
22                 <artifactId>maven-surefire-plugin</artifactId>
23                 <version>2.22.2</version>
24             </plugin>
25             <plugin>
26                 <artifactId>maven-failsafe-plugin</artifactId>
27                 <version>2.22.2</version>
28             </plugin>
29         </plugins>
30     </build>
31
32     <dependencies>
33         <dependency>
34             <groupId>org.junit.jupiter</groupId>
35             <artifactId>junit-jupiter-api</artifactId>
36             <version>${junit.jupiter.version}</version>
37             <scope>test</scope>
38         </dependency>
39         <dependency>
40             <groupId>org.junit.jupiter</groupId>
41             <artifactId>junit-jupiter-params</artifactId>
42             <version>${junit.jupiter.version}</version>
43             <scope>test</scope>
44         </dependency>
45         <dependency>
46             <groupId>org.junit.jupiter</groupId>
47             <artifactId>junit-jupiter-engine</artifactId>
48             <version>${junit.jupiter.version}</version>
49             <scope>test</scope>
50         </dependency>
51     </dependencies>
52 </project>
```

Fonte: Elaborada pelo autor

Vamos agora criar nosso exemplo...

1.3 Codificando nosso exemplo

Vamos criar uma classe chamada `Vector.java` no pacote `br.unesp.rc.junit5tutorial` e implementar dois métodos. O primeiro recebe como parâmetro dois vetores do tipo `int` e retorna se estes são iguais. Já o segundo método, recebe dois números inteiros, que representa o tamanho dos vetores, e retorna se os mesmos são iguais. Na **Listagem 4** é apresentado o código fonte da classe `Vector.java`.

Listagem 4 – Classe Vector.java

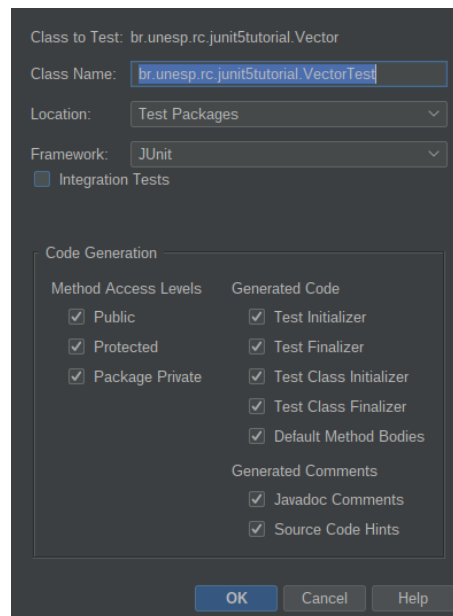
```
1 package br.unesp.rc.junit5tutorial;
2
3 /**
4  * Exemplo extraído do tutorial "Escrevendo Testes JUnit no NetBeans IDE"
5  * URL: https://netbeans.apache.org/kb/docs/java/junit-intro\_pt\_BR.html
6  * @author Prof. Frank J. Affonso
7  */
8 public class Vector {
9
10     private Vector() {
11     }
12
13     /**
14      * Verifica se os vetores são iguais
15      */
16     public static boolean equal(int[] a, int[] b) {
17         if ((a == null) || (b == null)) {
18             throw new IllegalArgumentException("null argument");
19         }
20         if (a.length != b.length) {
21             return false;
22         }
23         for (int i = 0; i < a.length; i++) {
24             if (a[i] != b[i]) {
25                 return false;
26             }
27         }
28         return true;
29     }
30
31     /**
32      * Verifica se os tamanhos dos vetores são iguais.
33      */
34     public static boolean size(int a, int b) {
35         if ((a < 0) || (b < 0)) {
```

```
36         throw new IllegalArgumentException("Valores negativos não são  
           ↳ permitidos!");  
37     }  
38     return a == b;  
39 }  
40 }
```

Fonte: Elaborada pelo autor

O próximo passo é criar os casos de testes para verificar se o código desenvolvido realmente funciona. Para isso, vamos aproveitar um recurso interessante da IDE Netbeans, que é a geração automática dos testes. Assim, clique no nome da classe e navegue pelas opções: `Navigate -> Go to test/Tested class` ou pelo atalho `Ctrl+Alt+T`. Na tela que se abre deixe marcada as seguintes opções, como ilustra a **Figura 2**.

Figura 2 – Tela para configuração dos testes

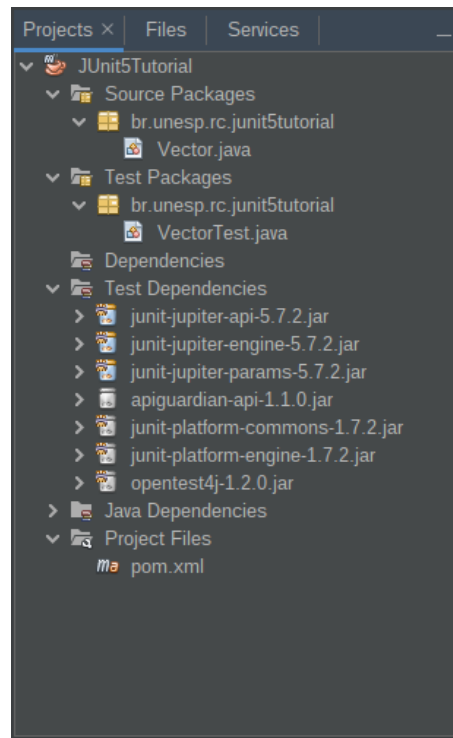


Fonte: Elaborada pelo autor

Veja a nova estrutura do projeto na **Figura 3**. Observe que a classe `VectorTest.java` foi gerada no pacote `br.unesp.rc.junit5tutorial` em `Test Packages`. Na **Listagem 5** é apresentado o código fonte completo dessa classe.

Listagem 5 – Classe VectorTest.java

```
1 package br.unesp.rc.junit5tutorial;  
2  
3 import org.junit.jupiter.api.AfterEach;  
4 import org.junit.jupiter.api.AfterAll;  
5 import org.junit.jupiter.api.BeforeEach;  
6 import org.junit.jupiter.api.BeforeAll;  
7 import org.junit.jupiter.api.Test;
```

Figura 3 – Estrutura do projeto

Fonte: Elaborada pelo autor

```
8  import static org.junit.jupiter.api.Assertions.*;
9
10 /**
11  * @author Prof. Frank J. Affonso
12  */
13 public class VectorTest {
14
15     public VectorTest() {
16     }
17
18     @BeforeAll
19     public static void setUpClass() {
20     }
21
22     @AfterAll
23     public static void tearDownClass() {
24     }
25
26     @BeforeEach
27     public void setUp() {
28     }
29
30     @AfterEach
```

```
31     public void tearDown() {
32     }
33
34     /**
35      * Test of equal method, of class Vector.
36      */
37     @Test
38     public void testEqual() {
39         System.out.println("equal");
40         int[] a = null;
41         int[] b = null;
42         boolean expectedResult = false;
43
44         boolean result = Vector.equal(a, b);
45         assertEquals(expResult, result);
46         // TODO review the generated test code and remove the default call
47         //    ↪ to fail.
48         fail("The test case is a prototype.");
49     }
50
51     /**
52      * Test of size method, of class Vector.
53      */
54     @Test
55     public void testSize() {
56         System.out.println("size");
57         int a = 0;
58         int b = 0;
59         boolean expectedResult = false;
60
61         boolean result = Vector.size(a, b);
62         assertEquals(expResult, result);
63         // TODO review the generated test code and remove the default call
64         //    ↪ to fail.
65         fail("The test case is a prototype.");
66     }
```

Fonte: Elaborada pelo autor

Agora vamos “colocar a mão na massa” e implementar os testes na classe `VectorTest.java` para verificar se o código elaborado em `Vector.java` realmente funciona. Visando simplificar a apresentação deste documento, a implementação da classe de teste `VectorTest.java` é apresentada na [Listagem 6](#). Em seguida, uma descrição detalhada dos trechos de código fonte utilizados

para elaboração dos casos de testes e das anotações para execução dos mesmos é reportada.

Listagem 6 – Classe VectorTest.java implementada

```
1 package br.unesp.rc.junit5tutorial;
2
3 import org.junit.jupiter.api.AfterEach;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6 import static org.junit.jupiter.api.Assertions.*;
7 import org.junit.jupiter.api.MethodOrderer;
8 import org.junit.jupiter.api.Order;
9 import org.junit.jupiter.api.TestMethodOrder;
10
11 /**
12  * @author Prof. Frank J. Affonso
13  */
14 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
15 public class VectorTest {
16
17     private int v1[];
18     private int v2[];
19
20     public VectorTest() {
21     }
22
23     @BeforeEach
24     public void setup() {
25         System.out.println("Iniciando os vetores");
26         int[] a = {1, 2, 3};
27         int[] b = {1, 2, 3};
28
29         this.v1 = new int[3];
30         this.v1 = a;
31         this.v2 = new int[3];
32         this.v2 = b;
33     }
34
35     @AfterEach
36     public void finalize() {
37         System.out.println("Anulando os vetores");
38         this.v1 = null;
39         this.v2 = null;
40     }
41
42     @Order(2)
```



```
43     @Test
44     public void testEqual() {
45         boolean expResult = true;
46         boolean result = Vector.equal(this.v1, this.v2);
47         assertEquals(expResult, result);
48     }
49
50     @Order(1)
51     @Test
52     public void testSize() {
53         boolean expResult = true;
54         boolean result = Vector.size(this.v1.length, this.v2.length);
55         assertEquals(expResult, result);
56     }
57 }
```

Fonte: Elaborada pelo autor

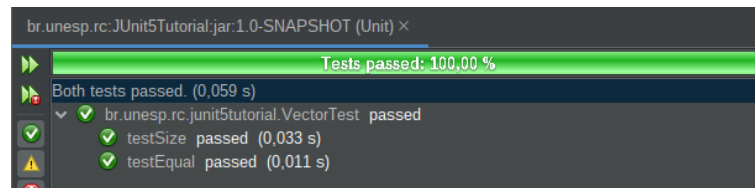
No que diz respeito ao código implementado, quatro métodos foram desenvolvidos. O método `setUp()`, Linhas 23 a 33, é responsável pela inicialização dos vetores. Observe que dois vetores IGUAIS de mesmo tamanho e mesmos números foram criados. Além disso, vale destacar que esse método foi anotado com `@BeforeEach`, que indica que o mesmo será chamado antes de cada execução de teste implementado na classe `VectorTest.java`. Já o método `finalize()`, Linhas 35 a 40, é responsável por anular os vetores. Nota-se também que esse método foi anotado com `@AfterEach`, que, de modo análogo ao anterior, será chamado para cada teste, só que após a execução. Os métodos `testEqual()`, Linhas 42 a 48, e `testSize()`, Linhas 50 a 56, representam os testes para a classe `Vector.java`, permitindo que sejam verificados os vetores quanto a igualdade e tamanho, respectivamente. Sobre esses dois últimos métodos vale destacar a presença das anotações `@Order`⁵ e `@Test`, que indicam a ordem de execução e a identificação do método como teste a ser executado, respectivamente. Por fim, vale informar que é necessário anotar a classe `VectorTest.java` com `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` para que as anotações de ordem de execução funcionem corretamente.

1.4 Executando os testes

Para executar os testes clique sobre a classe `VectorTest.java` com o botão direito e, em seguida, clique em `Test File`. Na **Figura 4** é apresentada a tela de execução dos testes, onde é possível observar que os testes foram executados corretamente (linha verde e verificadores verdes).

Sobre o exemplo apresentado (**Figura 4**), vale destacar que mais casos de testes devem ser elaborados para comprovar que a implementação está correta. Neste documento foi apresentado

⁵ O número entre parênteses representa a ordem de execução do teste.

Figura 4 – Resultado da execução

Fonte: Elaborada pelo autor

apenas um caso de teste! Aquele que funciona!!!! **Cuidado para não gerar apenas o caso de teste que funcione!** Lembre-se que objetivo deste documento é apresentar o funcionamento do *framework* JUnit 5 em uma IDE de desenvolvimento (Netbeans).

Para finalizar, vale informar que a anotação `@Disabled` pode ser inserida acima de um método para que o mesmo não seja executado. Como essa operação é muito simples, a mesma não será apresentada neste documento. Fica como teste para o leitor!

1.5 Suíte de teste

Para apresentar como implementar uma suíte de testes, a versão 4.13.2 será utilizada. A justificativa para *downgrade* de versão está na incompatibilidade da versão 5.X entre as IDEs de desenvolvimento. De acordo com pesquisas realizadas nos melhores fórum voltados para este tipo de atividade, esse problema ainda não foi solucionado⁶ e o pessoal está recomendando o uso da versão 4.X.

No que diz respeito ao desenvolvimento, o exemplo anterior será utilizado. Para mostrar o funcionamento de uma suíte de teste, a classe `VectorTest.java` será refatorada e duas classes serão criadas, a saber: `VectorEqualTest` e `VectorSizeTest`. Essas classes visam testar se dois vetores são iguais em conteúdo e tamanho, respectivamente. A **Listagem 7** mostra a listagem completa da classe `VectorEqualTest.java`. Observe que as anotações (`@Before` e `@After`) foram modificadas em relação ao exemplo anterior (**Listagem 6**).

Listagem 7 – Classe VectorEqualTest.java

```
1 package br.unesp.rc.junit4tutorial;
2
3 import org.junit.After;
4 import org.junit.Before;
5 import org.junit.Test;
6 import static org.junit.Assert.*;
7
8 /**
9  *
```

⁶ Data de redação deste documento: Julho de 2021. Atualização em Agosto de 2022.

```
10  * @author Prof. Frank J. Affonso
11  */
12  public class VectorEqualTest {
13
14      private int v1[];
15      private int v2[];
16
17      public VectorEqualTest() {
18      }
19
20      @Before
21      public void setUp() {
22          System.out.println("Inicializando os vetores");
23          int[] a = {1, 2, 3};
24          int[] b = {1, 2, 3};
25
26          this.v1 = new int[3];
27          this.v1 = a;
28          this.v2 = new int[3];
29          this.v2 = b;
30      }
31
32      @After
33      public void tearDown() {
34          System.out.println("Anulando os vetores");
35          this.v1 = null;
36          this.v2 = null;
37      }
38
39      @Test
40      public void testEqual() {
41          System.out.println("Test: testEqual");
42          boolean expectedResult = true;
43          boolean result = Vector.equal(this.v1, this.v2);
44          assertEquals(expectedResult, result);
45      }
46  }
```

Fonte: Elaborada pelo autor

A **Listagem 8** mostra a listagem da classe `VectorSizeTest.java`. Observe que o trecho de código dos métodos `setUp()` e `tearDown()` foram omitidos por serem idênticos ao código apresentado na **Listagem 7**.

Listagem 8 – Classe `VectorSizeTest.java`

```
1  package br.unesp.rc.junit4tutorial;
```

```
2
3 import org.junit.After;
4 import static org.junit.Assert.assertEquals;
5 import org.junit.Before;
6 import org.junit.Test;
7
8 /**
9  *
10  * @author Prof. Frank J. Affonso
11  */
12 public class VectorSizeTest {
13
14     private int v1[];
15     private int v2[];
16
17     public VectorSizeTest() {
18     }
19
20     @Before
21     public void setUp() {
22         // corpo do método omitido (ver Listagem 7)
23     }
24
25     @After
26     public void tearDown() {
27         // corpo do método omitido (ver Listagem 7)
28     }
29
30     @Test
31     public void testSize() {
32         System.out.println("Test: testSize");
33         boolean expResult = true;
34         boolean result = Vector.size(this.v1.length, this.v2.length);
35         assertEquals(expResult, result);
36     }
37 }
```

Fonte: Elaborada pelo autor

Agora podemos criar uma suíte de teste para que os testes elaborados anteriormente (classes `VectorEqualTest` e `VectorSizeTest`) sejam executados. Para isso, vamos criar uma classe de teste chamada `VectorTest`, como indicado no início deste tutorial. A [Listagem 9](#) mostra a listagem completa dessa classe. Basicamente, uma suíte teste é indicada pela anotação `@Suite.SuiteClasses` e os valores que essa anotação possui. Observe que a tag `value=...` possui a lista de classes de teste que serão executadas `VectorEqualTest.class` e `VectorSizeTest.class`.

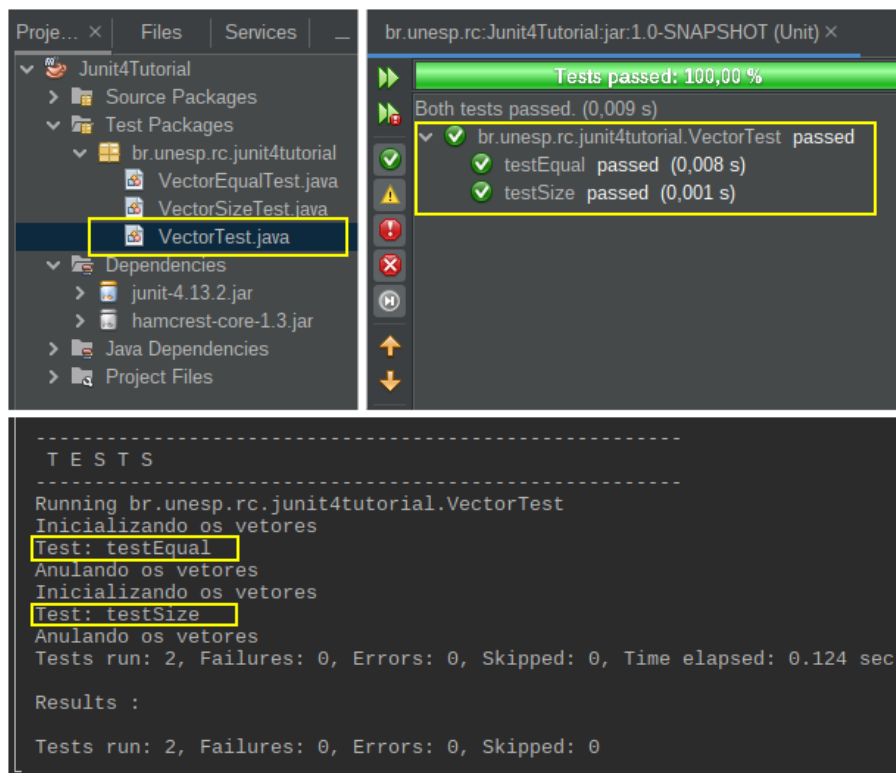
Listagem 9 – Classe VectorTest.java implementada

```
1 package br.unesp.rc.junit4tutorial;
2
3 import org.junit.runner.RunWith;
4 import org.junit.runners.Suite;
5 /**
6  *
7  * @author Prof. Frank J. Affonso
8  */
9 @RunWith(Suite.class)
10 @Suite.SuiteClasses(value = {VectorEqualTest.class, VectorSizeTest.class})
11 public class VectorTest {
12
13 }
```

Fonte: Elaborada pelo autor

A **Figura 5** mostra a estrutura do projeto desenvolvido com o JUnit 4 e os resultados de execução em modo gráfico e texto. A execução da suíte de teste segue o mesmo procedimento de um teste de unidade, conforme apresentado na **Seção 1.4**. Na parte inferior dessa figura é possível observar as chamadas dos testes executados para validação do tamanho e conteúdo, ambos identificado pela retângulo em amarelo.

Figura 5 – Projeto e execução dos testes



Fonte: Elaborada pelo autor

Para finalizar este documento com uma atividade interessante, modifique os testes de unidade (classes `VectorEqualTest` e `VectorSizeTest`) e execute a suíte (classe `VectorTest`) novamente para visualizar o resultado.

Parte II

Aplicações Java com o *framework* Spring Data

2 Desenvolvendo aplicações Java com Spring Data JPA

Autor(es):

Prof. Dr. Frank J. Affonso

Pré-requisitos:

Java: OpenJDK 64-Bit Server VM AdoptOpenJDK-11.0.11+9

Spring Boot: versão 2.5.2

Netbeans IDE: versão 12.3

MySQL Server: versão 8.0.26 *Community Server* - GPL

MySQL Workbench: versão 8.0.26 *Community*

Sistema Operacional: Linux - Kubuntu 20.04.1

O objetivo principal desta seção é apresentar aos interessados como o *framework* Spring pode ser utilizado na construção de aplicações Java que envolvem bases de dados relacionais. Para isso, o banco de dados MySQL Server e a ferramenta MySQL Workbench serão utilizados. No que diz respeito ao conhecimento teórico, os seguintes conceitos serão utilizados: organização do sistema em camadas, padrão de projeto DAO (*Data Access Object*, mapeamento objeto-relacional, e programação orientada a objetos em geral.)

2.1 Falando um pouco do Spring Framework

Em poucas palavras, Spring é um *framework* abrangente que fornece um modelo de programação e configuração para aplicativos corporativos modernos baseados em Java. Um elemento importante desse *framework* é o suporte de infraestrutura no nível do aplicativo, pois o Spring se concentra no “encanamento” das aplicações corporativas, abstraindo, automatizando e facilitando o desenvolvimento de tais aplicações. Dessa forma, as equipes de desenvolvimento podem direcionar seus esforços na lógica de negócios (SPRING, 2021b).

Dentre os projetos existentes no ecossistema Spring, esta seção abordará o Spring Data¹.

¹ <https://spring.io/projects/spring-data>

O principal objetivo deste projeto (Spring Data) é fornecer um modelo de programação baseado em Spring que permite o acesso a dados, mantendo as características especiais do armazenamento de dados subjacente. O Spring Data facilita o uso de tecnologias de acesso a dados, bancos de dados relacionais e não relacionais, estruturas de redução de mapas e serviços de dados baseados em nuvem. Dentre os módulos existentes, esta seção abordará o Spring Data JPA (do inglês, *Java Persistence API*).

O Spring Data JPA permite o desenvolvimento para camadas de acesso a dados baseadas em JPA, abstraindo e facilitando a codificação dessa camada. Dessa forma, pode-se dizer que esse módulo facilita a criação de aplicativos com tecnologia Spring que usam tecnologias de acesso a dados para banco de dados relacionais. De acordo com a documentação do projeto ([SPRING, 2021a](#)), as principais características podem ser sintetizadas em:

- Repositório poderoso e abstrações de mapeamento de objeto personalizadas;
- Derivação de consulta dinâmica a partir de nomes de métodos de repositório;
- Possibilidade de integrar código de repositório customizado;
- Integração avançada com outros controladores Spring;
- Entre outras

(TRADUÇÃO NOSSA)

2.2 Apresentando o exemplo

Para mostrar a aplicabilidade do *framework* Spring Data JPA, o seguinte exemplo será utilizado. Veja a especificação abaixo:

O sistema deve permitir a inclusão, alteração e remoção de clientes do sistema SPRING-DATA-JPA com os seguintes atributos: nome, endereço (rua, número, bairro, cep, cidade, estado), contato (telefone residencial, telefone comercial, celular, e-mail), cpf (para pessoa física), data de nascimento e cnpj (para pessoa jurídica). Um cliente pode ter mais de um endereço para correspondência. Um cliente deve ter apenas um conjunto de informações de contato.

Com base na especificação apresentada, um diagrama de classes UML foi elaborado na biblioteca PlantUML² utilizando a ferramenta *online* PlantText³. Na **Listagem 10** é apresentado o código fonte do diagrama ilustrado na **Figura 6**:

² <https://plantuml.com/class-diagram>

³ <https://www.planttext.com/>

Listagem 10 – Diagrama.puml

```
1 @startuml
2 skinparam defaultFontSize 12
3 skinparam shadowing false
4 skinparam ranksep 60
5 skinparam nodesep 60
6
7 class Endereco{
8     - String rua
9     - int numero
10    - String bairro
11    - String cep
12    - String cidade
13    - String estado
14    + Endereco()
15 }
16
17 class Contato{
18     - String telefoneResidencial
19     - String telefoneComercial
20     - String celular
21     - String email
22     + Contato()
23 }
24
25 class Acesso{
26     - String usuario
27     - String senha
28     + Acesso()
29 }
30
31 class Pessoa{
32     - String nome
33     + Pessoa()
34 }
35
36 class Fisica{
37     - Date dataNascimento
38     - String cpf
39     + Fisica()
40 }
41
42 class Juridica{
43     - String cnpj
```

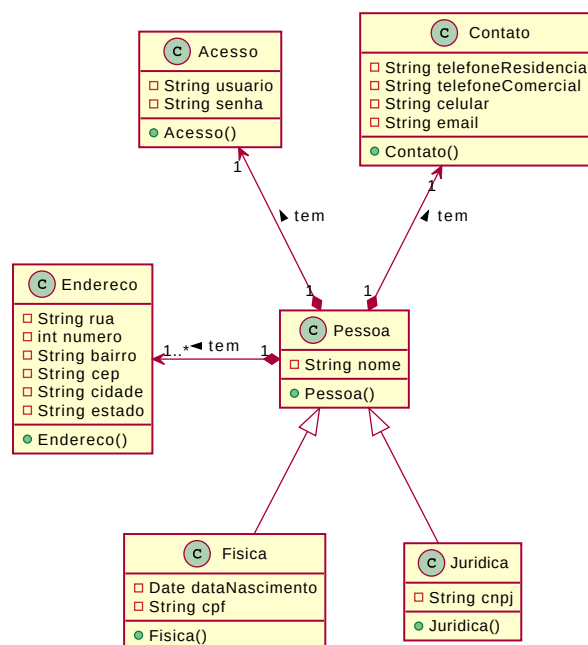
```

44     + Juridica()
45 }
46
47 Pessoa <|-- Fisica
48 Pessoa <|-- Juridica
49 Pessoa "1" *-up-> "1" Contato: tem >
50 Pessoa "1" *-left-> "1..*" Endereco: tem >
51 Pessoa "1" *-up-> "1" Acesso: tem >
52
53 @enduml

```

Fonte: Elaborada pelo autor

Figura 6 – Diagrama de Classes UML



Fonte: Elaborada pelo autor

Como o *framework* Spring Data JPA permite que o mapeamento entre o modelo orientado a objetos (Figura 6) e o banco de dados relacional seja realizado por meio de anotações, apenas os conceitos aplicáveis aos relacionamento desse modelo serão explorados neste documento.

2.3 Criando o projeto

No site oficial⁴ do *framework* Spring são disponibilizados os seguintes recursos aos desenvolvedores:

- **Spring Too Suite 4**, uma ferramenta baseada na IDE Eclipse para desenvolvimento de aplicações baseadas em seu ecossistema;

⁴ <<https://spring.io/>>

- **Spring initializr**, um website (<<https://start.spring.io/>>) que permite a criação de projetos.
- **NOTA.** IDEs de desenvolvimento como Netbeans e IntelliJ, assim como editores como VSCode, permitem instalação de *plugins* para trabalhar com o *framework*.

Para o desenvolvimento do exemplo apresentado na **Seção 2.2**, a segunda opção (**Spring initializr**) será utilizada. Sendo assim, acesse o website indicado na descrição da mesma e configure um projeto como ilustrado na **Figura 7**.

Figura 7 – Diagrama de Classes UML

The screenshot shows the Spring Initializr website interface. The browser address bar displays <https://start.spring.io>. The page features several configuration sections, some of which are highlighted with red boxes:

- Project:** Includes radio buttons for **Maven Project** (selected) and **Gradle Project**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for versions **2.6.0 (SNAPSHOT)**, **2.5.3 (SNAPSHOT)**, **2.5.2** (selected), **2.4.9 (SNAPSHOT)**, and **2.4.8**.
- Project Metadata:** A form with fields for Group (br.unesp.rc), Artifact (springtutorial), Name (springtutorial), Description (Demo project for Spring Boot), and Package name (br.unesp.rc.springtutorial). It also includes Packaging options (Jar selected, War) and Java version options (16, 11 selected, 8).
- Dependencies:** A list of dependencies with red minus buttons for removal:
 - Spring Boot DevTools** (DEVELOPER TOOLS)
 - Lombok** (DEVELOPER TOOLS)
 - Spring Data JPA** (SQL)
 - MySQL Driver** (SQL)
 - Spring Web** (WEB)

At the bottom of the page, there are three buttons: **GENERATE** (CTRL + ↵), **EXPLORE** (CTRL + SPACE), and **SHARE...**.

Fonte: Elaborada pelo autor

IMPORTANTE: Observe as áreas destacadas pelas linhas em vermelho para configurar o projeto. Em relação aos módulos de dependência, o **Spring Boot DevTools** foi selecionado para que as alterações de código sejam refletidas no projeto em tempo de execução, sem que a aplicação tenha que ser reinicializada. O **Lombok** foi selecionado para evitar que código, como métodos de acesso, sejam escritos nas classes. O **Spring Data JPA** foi selecionado para que a persistência de dados (API JPA) com o banco de dados relacional MySQL seja

implementada. Por esse motivo que o **MySQL Driver** também foi selecionado. Por fim, já pensando um pouco no futuro, o Spring Web foi selecionado para que uma API RESTful possa ser implementada.

Ao clicar no botão **GENERATE**, um arquivo de extensão **.zip** será gerado e disponibilizado para download. Salve esse arquivo em uma pasta local de sua máquina para que o mesmo possa ser utilizado no desenvolvimento do exemplo.

2.4 Implementação do exemplo

2.4.1 Preparação do projeto

Para iniciarmos o projeto na IDE Netbeans 12.3, os seguintes passos devem ser executados:

- Criação de um **database** chamado **springtutorialjpa** na ferramenta MySQL Workbench. Por se tratar de uma operação elementar, os passos não serão apresentados neste documento.
- Descompacte o arquivo **springtutorial.zip** em um local de sua preferência. Abra o projeto na IDE Netbeans 12.3. Em seguida, abra o arquivo **application.properties** que está na pasta **Other Sources**. Vá navegando pelas pastas até encontrá-lo e insira as informações contidas na **Listagem 11**. Em relação ao conteúdo apresentado nessa listagem, a tag **spring.jpa.database-platform** deve ser destacada, pois o valor **org.hibernate.dialect.MySQL5InnoDBDialect** indica que *engine* do tipo **InnoDB** será utilizada. Esta *engine* permite utilização de recursos como integridade referencial, unicidade de campos, entre outros.

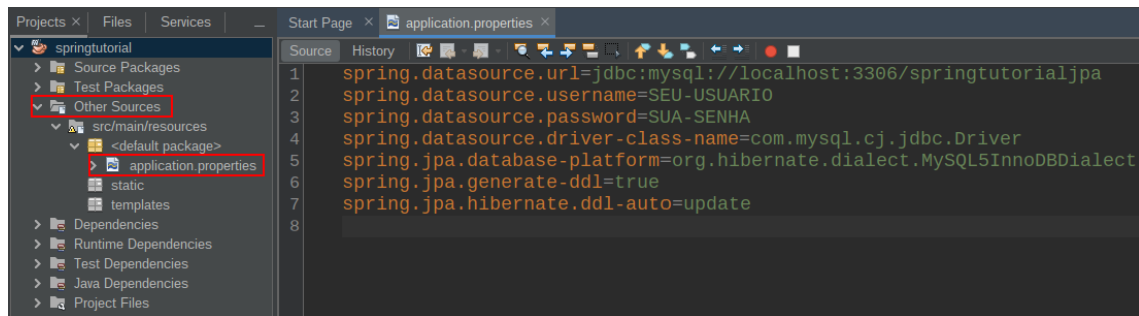
Listagem 11 – Arquivo application.properties

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/springtutorialjpa
2 spring.datasource.username=SEU-USUARIO
3 spring.datasource.password=SUA-SENHA
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.jpa.database-platform=org.hibernate.dialect.MySQL5InnoDBDialect
6 spring.jpa.generate-ddl=true
7 spring.jpa.hibernate.ddl-auto=update
```

Fonte: Elaborada pelo autor

A **Figura 8** ilustra a estrutura do projeto e o conteúdo do arquivo **application.properties**.

- Em seguida, clique no botão **Clear and Build Project** para que o download das dependências do projeto seja realizado. Se tudo der certo, a mensagem **BUILD SUCCESS** será exibida na aba de **Output** da IDE.

Figura 8 – Estrutura do projeto

Fonte: Elaborada pelo autor

2.4.2 Implementando a camada entity

Agora vamos começar a codificação das classes do nosso exemplo, conforme exposto no modelo UML (diagrama de classes) – **Figura 6**. Para isso, o *framework* **Lombok** será utilizado para minimizar a tarefa de codificação. Para isso, as classes serão anotadas com `@Getter`, `@Setter`, `@EqualsAndHashCode` e `@ToString`. Essas anotações permitem que os métodos *getters*, *setters*, *equals*, *hashCode* e *toString* sejam implementados em cada classe/entidade. Na **Listagem 12** é apresentado o código da classe `Acesso.java`.

Listagem 12 – Código fonte da classe Acesso.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import lombok.EqualsAndHashCode;
4 import lombok.Getter;
5 import lombok.Setter;
6 import lombok.ToString;
7
8 /**
9  * @author Prof. Dr. Frank J. Affonso
10  */
11 @Getter
12 @Setter
13 @EqualsAndHashCode
14 @ToString
15 public class Acesso {
16     private String usuario;
17     private String senha;
18
19     public Acesso() {
20     }
21 }
```

Fonte: Elaborada pelo autor

Na **Listagem 13** é apresentado o código da classe `Contato.java`.

Listagem 13 – Código fonte da classe Contato.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import lombok.EqualsAndHashCode;
4 import lombok.Getter;
5 import lombok.Setter;
6 import lombok.ToString;
7
8 /**
9  *
10  * @author Prof. Dr. Frank J. Affonso
11  */
12 @Getter
13 @Setter
14 @EqualsAndHashCode
15 @ToString
16 public class Contato {
17
18     private long idContato;
19     private String telefoneResidencial;
20     private String telefoneComercial;
21     private String celular;
22     private String email;
23
24     public Contato() {
25     }
26 }
```

Fonte: Elaborada pelo autor

Na **Listagem 14** é apresentado o código da classe `Endereco.java`.

Listagem 14 – Código fonte da classe Endereco.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import lombok.EqualsAndHashCode;
4 import lombok.Getter;
5 import lombok.Setter;
6 import lombok.ToString;
7
8 /**
9  *
10  * @author Prof. Dr. Frank J. Affonso
11  */
```

```
12 @Getter
13 @Setter
14 @EqualsAndHashCode
15 @ToString
16 public class Endereco {
17
18     private long idEndereco;
19     private String rua;
20     private int numero;
21     private String bairro;
22     private String cep;
23     private String cidade;
24     private String estado;
25
26     public Endereco() {
27     }
28 }
```

Fonte: Elaborada pelo autor

Na **Listagem 15** é apresentado o código da classe `Pessoa.java`.

Listagem 15 – Código fonte da classe Pessoa.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import lombok.EqualsAndHashCode;
6 import lombok.Getter;
7 import lombok.Setter;
8 import lombok.ToString;
9
10 /**
11  *
12  * @author Prof. Dr. Frank J. Affonso
13  */
14 @Getter
15 @Setter
16 @EqualsAndHashCode
17 @ToString
18 public class Pessoa {
19
20     private long idPessoa;
21     private String nome;
22     private List<Endereco> endereco;
23     private Acesso acesso;
```



```
24     private Contato contato;
25
26     public Pessoa() {
27         this.endereco = new ArrayList<>();
28     }
29
30     public void setEndereco(Endereco endereco) {
31         this.endereco.add(endereco);
32     }
33 }
```

Fonte: Elaborada pelo autor

Na **Listagem 16** é apresentado o código da classe `Fisica.java`. Observe que a anotação `@EqualsAndHashCode` teve o parâmetro `callSuper = false` adicionado para que os atributos da superclasse `Pessoa` não seja utilizado. Já a anotação `@ToString` teve dois parâmetros adicionados `callSuper=true`, `includeFieldNames=true` para que o método da superclasse seja chamado com a inclusão dos atributos.

Listagem 16 – Código fonte da classe Fisica.java

```
1  package br.unesp.rc.springtutorial.entity;
2
3  import java.util.Date;
4  import lombok.EqualsAndHashCode;
5  import lombok.Getter;
6  import lombok.Setter;
7  import lombok.ToString;
8
9  /**
10   *
11   * @author Prof. Dr. Frank J. Affonso
12   */
13  @Getter
14  @Setter
15  @EqualsAndHashCode(callSuper = false, exclude = {"dataNascimento"})
16  @ToString(callSuper=true, includeFieldNames=true)
17  public class Fisica extends Pessoa {
18
19     private String cpf;
20     private Date dataNascimento;
21
22     public Fisica() {
23     }
24 }
```

Fonte: Elaborada pelo autor

Na **Listagem 17** é apresentado o código da classe `Juridica.java`.

Listagem 17 – Código fonte da classe `Juridica.java`

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import lombok.EqualsAndHashCode;
4 import lombok.Getter;
5 import lombok.Setter;
6 import lombok.ToString;
7
8 /**
9  *
10  * @author Prof. Dr. Frank J. Affonso
11  */
12 @Getter
13 @Setter
14 @EqualsAndHashCode(callSuper = false)
15 @ToString(callSuper=true, includeFieldNames=true)
16 public class Juridica extends Pessoa {
17
18     private String cnpj;
19
20     public Juridica() {
21     }
22 }
```

Fonte: Elaborada pelo autor

Após anotar as classes/entidades, podemos executar o projeto para verificar o funcionamento das anotações inseridas. Para isso, a classe `SpringtutorialApplication.java` do pacote `br.unesp.rc.springtutorial` deve implementar a interface `CommandLineRunner` e o método `run` dessa interface deve ser reescrito. Na **Listagem 18** é apresentado o código da classe `SpringtutorialApplication.java`.

Listagem 18 – Código fonte da classe `SpringtutorialApplication.java`

```
1 package br.unesp.rc.springtutorial;
2
3 import br.unesp.rc.springtutorial.entity.Fisica;
4 import br.unesp.rc.springtutorial.utils.InstanceGenerator;
5 import org.springframework.boot.CommandLineRunner;
6 import org.springframework.boot.SpringApplication;
7 import org.springframework.boot.autoconfigure.SpringBootApplication;
8 import org.springframework.context.annotation.Bean;
9
10 /**
11  *
```

```

12  * @author Prof. Dr. Frank J. Affonso
13  */
14  @SpringBootApplication
15  public class SpringtutorialApplication implements CommandLineRunner {
16
17      public static void main(String[] args) {
18          SpringApplication.run(SpringtutorialApplication.class, args);
19      }
20
21      @Override
22      public void run(String... args) throws Exception {
23          Fisica entity = InstanceGenerator.getPessoaFisica("111.222.333-44",
24                  ↪ "user1");
25          System.out.println("\n" + entity + "\n");
26      }
27  }

```

Fonte: Elaborada pelo autor

Veja o resultado da execução (arquivo organizado) na [Listagem 19](#), que mostra os dados do objeto da classe `Fisica` instanciado:

Listagem 19 – Resultado da execução

```

1  Fisica(
2      super=Pessoa(idPessoa=0, nome=Juca Chaves,
3      endereco=[
4          Endereco(idEndereco=0, rua=Avenida 24A, numero=1515, bairro=Bela
5              ↪ Vista, cep=13506-900, cidade=Rio Claro, estado=SP),
6          Endereco(idEndereco=0, rua=Avenida 24A, numero=2020, bairro=Bela
7              ↪ Vista, cep=13506-900, cidade=Rio Claro, estado=SP),
8          Endereco(idEndereco=0, rua=Avenida 24A, numero=3030, bairro=Bela
9              ↪ Vista, cep=13506-900, cidade=Rio Claro, estado=SP)
10     ],
11     acesso=Acesso(usuario=user1, senha=deitel),
12     contato=Contato(idContato=0, telefoneResidencial=3333-4444,
13         ↪ telefoneComercial=3333-5555, celular=98888-7777,
14         email=juca@chaves.com.br)),
15     cpf=111.222.333-44, dataNascimento=Tue Jul 14 18:35:00 BRT 2020
16 )

```

Fonte: Elaborada pelo autor

2.4.3 Implementando a camada repository

Para que as classes de nosso exemplo possam ser persistidas no banco dados, anotações JPA devem inseridas em cada uma delas. Inicialmente, cada classe será modificada para implementar

a interface `java.io.Serializable` e declarar o atributo `serialVersionUID` (Linha 20), conforme recomendado pela especificação JavaBean (ORACLE, 2021). A classe `Acesso.java` foi anotada como `@Embeddable` (Linha 15) para ser tornar entidade embutida de `Pessoa`. Os atributos foram anotados com `@Column` (linhas 24 e 27) para que os referidos sejam inseridos na tabela `Pessoa`. Na Listagem 20 é apresentado o código da classe `Acesso.java`.

Listagem 20 – Código fonte da classe Acesso.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.io.Serializable;
4 import javax.persistence.Column;
5 import javax.persistence.Embeddable;
6 import lombok.EqualsAndHashCode;
7 import lombok.Getter;
8 import lombok.Setter;
9 import lombok.ToString;
10
11 /**
12  *
13  * @author Prof. Dr. Frank J. Affonso
14  */
15 @Embeddable
16 @Getter
17 @Setter
18 @ToString
19 @EqualsAndHashCode
20 public class Acesso implements Serializable {
21
22     private static final long serialVersionUID = 1L;
23
24     @Column(name = "acesso_usuario", unique = true)
25     private String usuario;
26
27     @Column(name = "acesso_senha")
28     private String senha;
29
30     public Acesso() {
31     }
32 }
```

Fonte: Elaborada pelo autor

A classe `Contato.java` foi anotada como `@Entity(name = "Contato")` (Linha 26) para definir o nome da tabela no banco de dados. O atributo `idContato` foi anotado com `@Id` e `@GeneratedValue (strategy = GenerationType.AUTO)` (linhas 26 a 28), que define o atributo

como chave no banco de dados do tipo autoincremento. Na **Listagem 21** é apresentado o código da classe `Contato.java`.

Listagem 21 – Código fonte da classe `Contato.java`

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.io.Serializable;
4 import javax.persistence.Entity;
5 import javax.persistence.GeneratedValue;
6 import javax.persistence.GenerationType;
7 import javax.persistence.Id;
8 import lombok.EqualsAndHashCode;
9 import lombok.Getter;
10 import lombok.Setter;
11 import lombok.ToString;
12
13 /**
14  *
15  * @author Prof. Dr. Frank J. Affonso
16  */
17 @Entity(name = "Contato")
18 @Getter
19 @Setter
20 @ToString
21 @EqualsAndHashCode
22 public class Contato implements Serializable {
23
24     private static final long serialVersionUID = 1L;
25
26     @Id
27     @GeneratedValue(strategy = GenerationType.AUTO)
28     private long idContato;
29
30     private String telefoneResidencial;
31     private String telefoneComercial;
32     private String celular;
33     private String email;
34
35     public Contato() {
36     }
37 }
```

Fonte: Elaborada pelo autor

Analogamente, a classe `Endereco.java` foi anotada com as mesmas anotações de `Contato.java`. Na **Listagem 22** é apresentado o código da classe `Endereco.java`.

Listagem 22 – Código fonte da classe Endereco.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.io.Serializable;
4 import javax.persistence.Entity;
5 import javax.persistence.GeneratedValue;
6 import javax.persistence.GenerationType;
7 import javax.persistence.Id;
8 import lombok.EqualsAndHashCode;
9 import lombok.Getter;
10 import lombok.Setter;
11 import lombok.ToString;
12
13 /**
14  *
15  * @author Prof. Dr. Frank J. Affonso
16  */
17 @Entity(name = "Endereco")
18 @Getter
19 @Setter
20 @ToString
21 @EqualsAndHashCode
22 public class Endereco implements Serializable {
23
24     private static final long serialVersionUID = 1L;
25
26     @Id
27     @GeneratedValue(strategy = GenerationType.AUTO)
28     private long idEndereco;
29
30     private String rua;
31     private int numero;
32     private String bairro;
33     private String cep;
34     private String cidade;
35     private String estado;
36
37     public Endereco() {
38     }
39 }
```

Fonte: Elaborada pelo autor

As classes de nível de hierarquia inferior, `Fisica.java` e `Juridica.java`, foram anotadas com `@Entity` e `@Table`. A primeira anotação indica que a classe será uma entidade no banco

de dados. Já a segunda, é utilizada para mapear informações entre a entidade (modelo lógico) e a tabela no banco de dados. Na **Listagem 23** é apresentado o código da classe `Fisica.java`. Observe, Linha 16, que `name = "PessoaFisica"` define o nome tabela para a classe supracitada. Além disso, vale destacar a anotação `@Column(name = "cpf", unique = true)` estabelece uma restrição de unicidade para o atributo `cpf`.

Listagem 23 – Código fonte da classe Fisica.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.util.Date;
4 import javax.persistence.Column;
5 import javax.persistence.Entity;
6 import javax.persistence.Table;
7 import javax.persistence.Temporal;
8 import javax.persistence.TemporalType;
9 import lombok.EqualsAndHashCode;
10 import lombok.Getter;
11 import lombok.Setter;
12 import lombok.ToString;
13
14 /**
15  * @author Prof. Dr. Frank J. Affonso
16  */
17 @Entity
18 @Table(name = "PessoaFisica")
19 @Getter
20 @Setter
21 @EqualsAndHashCode(callSuper = false, exclude = {"dataNascimento"})
22 @ToString(callSuper = true, includeFieldNames = true)
23 public class Fisica extends Pessoa {
24
25     private static final long serialVersionUID = 1L;
26
27     @Column(name = "cpf", unique = true)
28     private String cpf;
29
30     @Temporal(TemporalType.TIMESTAMP)
31     private Date dataNascimento;
32
33     public Fisica() {
34     }
35
36 }
```

A classe `Juridica.java` recebe as mesmas anotações da classe `Fisica.java`, como pode ser observado na **Listagem 24**. Observe o nome tabela (`name = "PessoaJuridica"`) definido na Linha 19 e a restrição de unicidade para o atributo `cnnpj` na Linha 28.

Listagem 24 – Código fonte da classe `Juridica.java`

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import javax.persistence.Column;
4 import javax.persistence.Entity;
5 import javax.persistence.Table;
6 import lombok.EqualsAndHashCode;
7 import lombok.Getter;
8 import lombok.Setter;
9 import lombok.ToString;
10
11 /**
12  *
13  * @author Prof. Dr. Frank J. Affonso
14  */
15 @Entity
16 @Table(name = "PessoaJuridica")
17 @Getter
18 @Setter
19 @EqualsAndHashCode(callSuper = false)
20 @ToString(callSuper = true, includeFieldNames = true)
21 public class Juridica extends Pessoa {
22
23     private static final long serialVersionUID = 1L;
24
25     @Column(name = "cnnpj", unique = true)
26     private String cnnpj;
27
28     public Juridica() {
29     }
30 }
```

Fonte: Elaborada pelo autor

Por fim, chegamos na classe mais “importante” do nosso exemplo, pelos atributos de relacionamento nela contidos (veja **Listagem 25**). O atributo `endereco` (Linha 47) foi anotado com duas anotações, a saber:

- `OneToMany` (linhas 42 a 45), indica que o atributo faz mapeamento de um para muitos com a classe `Endereco`.

- O significado de `CascadeType.ALL` é que a persistência deve ser propagada em cascata em todas as operações do `EntityManager` (`PERSIST`, `REMOVE`, `REFRESH`, `MERGE`, `DETACH`) e entidades relacionadas.
- O significado de `orphanRemoval = true` indica que as entidades do relacionamento que ficaram soltas após a entidade detentora ser excluída, também devem ser excluídas.
- O significado de `fetch = FetchType.EAGER` indica que o carregamento das entidades deve ser realizado, mesmo que estas não forem utilizadas.
- `@JoinColumn` (Linha 46), indica o nome do atributo de relacionamento (chave estrangeira) na tabela que possui o relacionamento muitos.

Observe que o atributo `acesso` (Linha 49) não foi anotado aqui por ter sido anotado como relacionamento embutido na **Listagem 20** – Linha 15. Já o atributo `contato` (Linha 53) foi anotado com duas anotações, a saber:

- `OneToOne` (Linha 51), indica que o atributo faz mapeamento de um para um com a classe `Contato`.
- `@JoinColumn` (Linha 52), indica o nome do atributo de relacionamento na tabela que representa a classe (entidade) em que ocorreu a anotação.

Listagem 25 – Código fonte da classe Pessoa.java

```
1 package br.unesp.rc.springtutorial.entity;
2
3 import java.io.Serializable;
4 import java.util.ArrayList;
5 import java.util.List;
6 import javax.persistence.CascadeType;
7 import javax.persistence.Entity;
8 import javax.persistence.FetchType;
9 import javax.persistence.GeneratedValue;
10 import javax.persistence.GenerationType;
11 import javax.persistence.Id;
12 import javax.persistence.Inheritance;
13 import javax.persistence.InheritanceType;
14 import javax.persistence.JoinColumn;
15 import javax.persistence.OneToMany;
16 import javax.persistence.OneToOne;
17 import lombok.EqualsAndHashCode;
18 import lombok.Getter;
19 import lombok.Setter;
20 import lombok.ToString;
```

```
21
22 /**
23  *
24  * @author Prof. Dr. Frank J. Affonso
25  */
26 @Entity(name = "Pessoa")
27 @Inheritance(strategy = InheritanceType.JOINED)
28 @Getter
29 @Setter
30 @EqualsAndHashCode
31 @ToString
32 public class Pessoa implements Serializable {
33
34     private static final long serialVersionUID = 1L;
35
36     @Id
37     @GeneratedValue(strategy = GenerationType.AUTO)
38     private long idPessoa;
39
40     private String nome;
41
42     @OneToMany(
43         cascade = CascadeType.ALL,
44         orphanRemoval = true,
45         fetch = FetchType.EAGER)
46     @JoinColumn(name = "pessoa_idpessoa")
47     private List<Endereco> endereco;
48
49     private Acesso acesso;
50
51     @OneToOne(cascade = CascadeType.ALL)
52     @JoinColumn(name = "contato_idContato")
53     private Contato contato;
54
55     public Pessoa() {
56         this.endereco = new ArrayList<>();
57     }
58
59     public void setEndereco(Endereco endereco) {
60         this.endereco.add(endereco);
61     }
62 }
```

Fonte: Elaborada pelo autor

Após anotar as classes (ou entidades) do modelo, podemos iniciar a implementação

da camada `repository`. Para isso, crie um pacote de mesmo nome da camada no pacote raiz (`br.unesp.rc.springtutorial`) de nosso exemplo. Em seguida, crie uma interface chamada `FisicaRepository`, que deve estender uma interface de repositório e, ao mesmo tempo, indicar o tipo de objeto que manipula e o tipo de chave (`JpaRepository<Fisica, Long>`). Na **Listagem 26** é apresentado o código da classe `FisicaRepository.java`. Observe que apenas uma assinatura foi elaborada (Linha 12). Vale ressaltar que essa assinatura segue o padrão do *framework* (`find + By + NOME-DO-CAMPO`).

Listagem 26 – Código fonte da classe `FisicaRepository.java`

```
1 package br.unesp.rc.springtutorial.repository;
2
3 import br.unesp.rc.springtutorial.entity.Fisica;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 /**
7  *
8  * @author Prof. Dr. Frank J. Affonso
9  */
10 public interface FisicaRepository extends JpaRepository<Fisica, Long> {
11
12     Fisica findByCpf(String cpf);
13
14 }
```

Fonte: Elaborada pelo autor

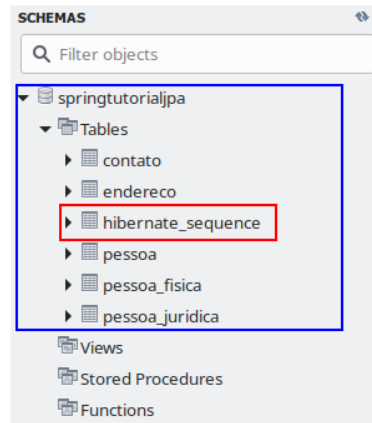
Após implementação do repositório, execute o projeto e observe o resultado. Se tudo estiver correto, o Spring Data JPA irá criar as tabelas no banco de dados (linha em azul), conforme ilustra a **Figura 9**. Observe também que a uma sequence chamada `hibernate_sequence` (linha vermelha) foi criada para gerenciar os `IDs` dos objetos. Lembre-se que as classes foram anotadas com (`@GeneratedValue(strategy = GenerationType.AUTO)`) para geração automática dos identificadores.

2.4.4 Implementando a camada service

Como forma de abstrair a camada de repositório do nosso exemplo, vamos iniciar a implementação da camada `service`. Para isso, vamos criar um pacote de mesmo nome da camada no pacote raiz (`br.unesp.rc.springtutorial`). Em seguida, crie uma classe chamada `FisicaService` para abstrair as operações do repositório. A listagem abaixo mostra o código fonte dessa classe. Na **Listagem 27** é apresentado o código da classe `FisicaService.java`. Observe que as operações básica para manipulação da entidade `Fisica`.

Listagem 27 – Código fonte da classe `FisicaService.java`

```
1 package br.unesp.rc.springtutorial.service;
```

Figura 9 – Estrutura do Esquema

Fonte: Elaborada pelo autor

```

2
3 import br.unesp.rc.springtutorial.entity.Fisica;
4 import br.unesp.rc.springtutorial.repository.FisicaRepository;
5 import java.util.ArrayList;
6 import java.util.List;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Component;
9
10 /**
11  *
12  * @author Prof. Dr. Frank J. Affonso
13  */
14 @Component
15 public class FisicaService {
16
17     @Autowired
18     private FisicaRepository repository;
19
20     public FisicaService() {
21     }
22
23     public Fisica save(Fisica entity) {
24         Fisica persistedEntity = null;
25
26         if (repository != null) {
27             persistedEntity = repository.save(entity);
28         }
29
30         return persistedEntity;
31     }

```

```
32
33     public Fisica findByCpf(String cpf) {
34         Fisica insertedEntity = null;
35
36         if (repository != null) {
37             insertedEntity = repository.findByCpf(cpf);
38         }
39
40         return insertedEntity;
41     }
42
43     public void delete(Fisica entity) {
44
45         if (repository != null) {
46             repository.delete(entity);
47         }
48     }
49
50     public Fisica update(Fisica entity) {
51
52         Fisica persistedEntity = null;
53
54         if (repository != null) {
55             persistedEntity = repository.save(entity);
56         }
57
58         return persistedEntity;
59     }
60
61     public List<Fisica> findAll() {
62         List<Fisica> list = null;
63
64         if (repository != null) {
65             list = new ArrayList<>();
66             list = repository.findAll();
67         }
68
69         return list;
70     }
71 }
```

Fonte: Elaborada pelo autor

No que diz respeito ao código apresentado na [Listagem 27](#), duas anotações devem ser detalhadas:

- `@Component`, esta anotação serve para indicar ao *framework* que classe é um *bean* que deverá ser gerenciado pela implementação de IoC (do inglês, *Inversion of Control*) *Container* do *framework* Spring.
- `@Autowired`, esta anotação indica ao *framework* que o atributo foi injetado na classe e deve ser por ele gerenciado.

Por fim, vale destacar que o desenvolvimento dessa camada mostra que os métodos fornecidos pelo repositório foram utilizados.

2.5 Escrevendo os casos de teste

Uma vez concluída a implementação das camadas, podemos trabalhar na implementação dos casos de testes para a classe `FisicaService`. Para isso, clique no nome da referida classe e navegue pelas opções: `Navigate -> Go to test/Tested class` ou pelo atalho `Ctrl+Alt+T`. Aceite as opções apresentadas e clique em `OK`. Altere o corpo dos métodos de teste conforme a **Listagem 28**. Observe nessa listagem que a classe foi anotada com `@SpringBootTest` (Linha 17) para indicar ao *framework* que essa classe deve ser por ele gerenciada. Além disso, vale destacar também a presença da anotação `@Autowired` no atributo `fs` (linhas 22 e 23). Observe, linhas 29 e 44, a criação da entidade *entity* pela classe `InstanceGenerator`, a qual permite gerar uma instância de “Pessoa Física” pelo método `getPessoaFisica`. A **Listagem 29** exibe o código fonte da classe `InstanceGenerator.java`.

Listagem 28 – Código fonte da classe `FisicaServiceTest.java`

```
1 package br.unesp.rc.springtutorial.service;
2
3 import br.unesp.rc.springtutorial.entity.Fisica;
4 import br.unesp.rc.springtutorial.utils.InstanceGenerator;
5 import java.util.List;
6 import org.junit.jupiter.api.Test;
7 import static org.junit.jupiter.api.Assertions.*;
8 import org.junit.jupiter.api.Disabled;
9 import org.junit.jupiter.api.DisplayName;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.boot.test.context.SpringBootTest;
12
13 /**
14  *
15  * @author Prof. Dr. Frank J. Affonso
16  */
17 @SpringBootTest
18 public class FisicaServiceTest {
```

```
19
20     private Fisica entity;
21
22     @Autowired
23     private FisicaService fs = new FisicaService();
24
25     @Disabled
26     @Test
27     @DisplayName("FisicaService.save(Fisica)")
28     void testSave() {
29         entity = InstanceGenerator.getPessoaFisica("222.333.444-55",
30             ↪ "user1");
31         System.out.println(entity);
32
33         Fisica f = fs.save(entity);
34         System.out.println("-----");
35         System.out.println(f);
36         System.out.println("-----");
37
38         assertEquals(entity, f);
39     }
40
41     @Disabled
42     @Test
43     @DisplayName("FisicaService.findByCpf(cpf)")
44     void testFindByCpf() {
45         entity = InstanceGenerator.getPessoaFisica("222.333.444-55",
46             ↪ "user1");
47
48         String cpf = "222.333.444-55";
49         Fisica f = fs.findByCpf(cpf);
50         System.out.println("-----");
51         System.out.println(f);
52         System.out.println("-----");
53
54         assertEquals(entity, f);
55     }
56
57     @Disabled
58     @Test
59     public void testDelete() {
60         //...
```

```
61     @Disabled
62     @Test
63     public void testUpdate() {
64         //...
65     }
66
67     @Disabled
68     @Test
69     @DisplayName("FisicaService.findAll()")
70     public void testFindAll() {
71         System.out.println("findAll");
72
73         Fisica expResult = null;
74         List<Fisica> result = fs.findAll();
75         for (Fisica f : result){
76             System.out.println("FISICA: " + f);
77         }
78         assertEquals(expResult, result);
79     }
80 }
```

Fonte: Elaborada pelo autor

Listagem 29 – Código fonte da classe InstanceGenerator.java

```
1  package br.unesp.rc.springtutorial.utils;
2
3  import br.unesp.rc.springtutorial.entity.Acesso;
4  import br.unesp.rc.springtutorial.entity.Contato;
5  import br.unesp.rc.springtutorial.entity.Endereco;
6  import br.unesp.rc.springtutorial.entity.Fisica;
7  import java.util.Date;
8
9  /**
10   *
11   * @author Prof. Dr. Frank J. Affonso
12   */
13  public class InstanceGenerator {
14
15      private InstanceGenerator() {
16
17      }
18
19      public static Fisica getPessoaFisica(String cpf, String user) {
20          Fisica f = new Fisica();
21      }
```



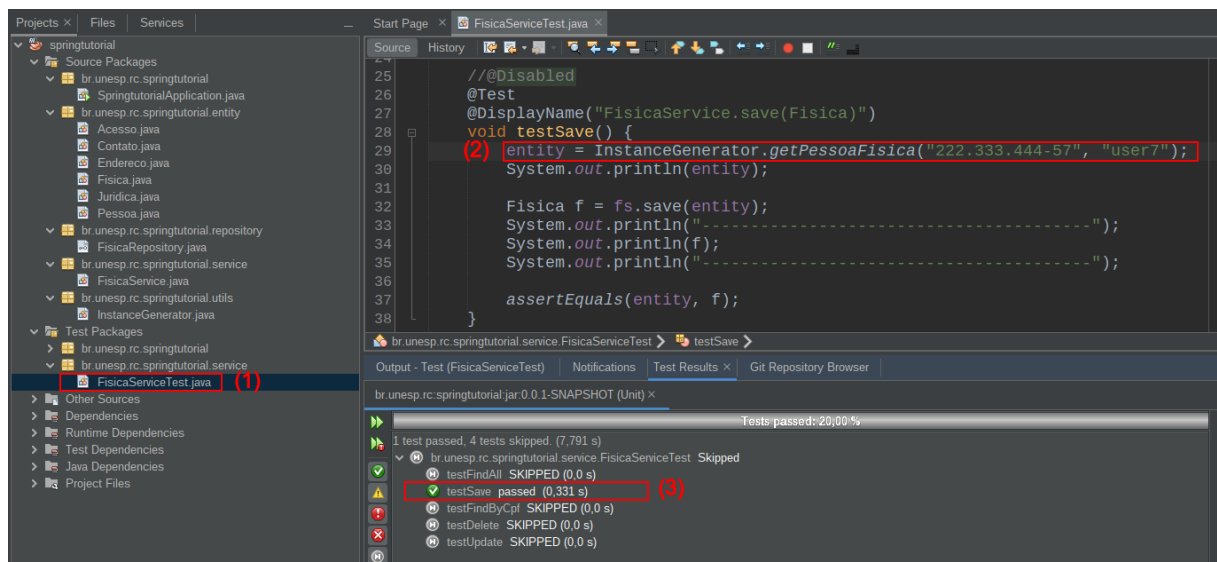
```
22     f.setCpf(cpf);
23     f.setDataNascimento(new Date());
24     f.setNome("Juca Chaves");
25
26     Acesso a = new Acesso();
27     a.setUsuario(user);
28     a.setSenha("deitel");
29     f.setAcesso(a);
30
31     Contato c = new Contato();
32     c.setTelefoneResidencial("3333-4444");
33     c.setTelefoneComercial("3333-5555");
34     c.setCelular("98888-7777");
35     c.setEmail("juca@chaves.com.br");
36     f.setContato(c);
37
38     Endereco e1 = new Endereco();
39     e1.setRua("Avenida 24A");
40     e1.setNumero(1515);
41     e1.setBairro("Bela Vista");
42     e1.setCep("13506-900");
43     e1.setCidade("Rio Claro");
44     e1.setEstado("SP");
45
46     Endereco e2 = new Endereco();
47     e2.setRua("Avenida 24A");
48     e2.setNumero(2020);
49     e2.setBairro("Bela Vista");
50     e2.setCep("13506-900");
51     e2.setCidade("Rio Claro");
52     e2.setEstado("SP");
53
54     Endereco e3 = new Endereco();
55     e3.setRua("Avenida 24A");
56     e3.setNumero(3030);
57     e3.setBairro("Bela Vista");
58     e3.setCep("13506-900");
59     e3.setCidade("Rio Claro");
60     e3.setEstado("SP");
61
62     f.setEndereco(e1);
63     f.setEndereco(e2);
64     f.setEndereco(e3);
65
```

```
66         return f;
67     }
68 }
```

Fonte: Elaborada pelo autor

Em relação a execução dos testes, observe que todos foram anotados com `@Disabled` para indicar ao *framework* que estão desabilitados. Portanto, comente a linha que contém a anotação `@Disabled` do teste a ser realizado. Por exemplo, para executar o teste que salva uma pessoa física (ou seja, método `testSave()` – Linha 28) no banco de dados, a Linha 25 deve ser comentada. A Figura 10 ilustra a execução do teste supracitado. Em (1) pode-se observar a classe de teste que será executada. Já em (2) nota-se que uma instância está sendo gerada com os atributos que necessitam de unicidade. Por fim, em (3) pode-se observar o resultado da execução do teste.

Figura 10 – Execução do teste



Fonte: Elaborada pelo autor

Sugestões para execução dos teste:

1. Execute um teste para inserir uma pessoa física já cadastrada;
2. Execute um teste para consulta de uma pessoa física existente;
3. Execute um teste para consulta de uma pessoa física não existente;
4. Execute um teste para listar todas as pessoas físicas existentes no banco de dados;
5. Implemente os casos de teste para permitir exclusão e alterações de pessoas físicas; e
6. Delete todas as pessoas e execute novamente o teste de listagem (Item 4).

Observe se a implementação realizada está correta!

2.6 Considerações finais

O Spring Data é um *framework* muito poderoso e possui vários outros recursos que não foram explorados neste documento. Espera-se que o conteúdo apresentado possa estimular os interessados no aprendizado e utilização em sistemas de maior escala.

Antes de começar a leitura da próxima seção é recomendado que se faça uma cópia do projeto ou que se utilize um gerenciamento via repositório (por exemplo, GitHub⁵, GitLab⁶). Para evitar problema futuros com sobreposição de código e retrabalho, o projeto gerado ao final deste documento será renomeado para `springDataJPA`. Apenas não se esqueça de utilizar o mesmo nome para `Name`, `ArtifactID` e `Folder`.

⁵ <<https://github.com/>>

⁶ <<https://about.gitlab.com/>>

Parte III

APIs RESTful com o *framework* Spring

3 Desenvolvendo APIs RESTful com Spring Web

Autor(es):

Prof. Dr. Frank J. Affonso

Pré-requisitos:

Java: OpenJDK 64-Bit Server VM AdoptOpenJDK-11.0.11+9

Spring Boot: versão 2.5.2

Netbeans IDE: versão 12.3

MySQL Server: versão 8.0.26 *Community Server* - GPL

MySQL Workbench: versão 8.0.26 *Community*

Postman: versão 7.36.4

Sistema Operacional: Linux - Kubuntu 20.04.1

3.1 Considerações iniciais

Antes de iniciar a apresentação desta seção, uma importante consideração deve ser destacada. Para que os interessados executem os passos aqui apresentados para construção das APIs, o desenvolvimento da aplicação reportado no **Capítulo 2** deve ser realizado. Portanto, pode-se dizer que o desenvolvimento do exemplo será realizado em continuidade, sendo reportados aqui os passos para adaptação da codificação existente e/ou criação de novas funcionalidades.

3.2 Implementando a camada resource

Antes de iniciar a codificação desta camada, edite o arquivo `application.properties` e adicione a entrada (`server.port=8084`). Essa entrada indica o número da porta HTTP (do inglês, *Hypertext Transfer Protocol*) que nossa API irá responder.

Para dar início ao desenvolvimento da camada `resource`, crie um pacote de mesmo nome no pacote principal `br.unesp.rc.springtutorial` de nosso exemplo. Em seguida, crie uma classe

chamada `FisicaResource` com o atributo chamado `fisicaService` da classe `FisicaService`. Anote esse atributo com `@Autowired` para dizer ao *framework* que este será por ele gerenciado. Para tornar a classe `FisicaResource` uma API, as seguintes anotações devem ser inseridas antes do nome da classe:

- `@RestController`, que indica que nossa classe é um *controller*; e
- `@RequestMapping("/entidade/v1")`, que indica como nossa API será acessada. Onde `entidade` é o nome do nosso exemplo e `v1` a versão da nossa API.

Na **Listagem 30** é apresentada a implementação da classe `FisicaResource` com o método `getAllFisica()` (linhas 22 a 26), que permite listar todas as entidades (pessoa física) existentes no banco de dados. Observe que esse método foi anotado com `@GetMapping("/")`, indicando que o mesmo responde diretamente pela URL da aplicação (`http://localhost:8084/entidade/v1/`).

Listagem 30 – Código fonte da classe FisicaResource.java

```
1 package br.unesp.rc.springtutorial.resource;
2
3 import br.unesp.rc.springtutorial.entity.Fisica;
4 import br.unesp.rc.springtutorial.service.FisicaService;
5 import java.util.List;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
11 /**
12  *
13  * @author Prof. Dr. Frank J. Affonso
14  */
15 @RestController
16 @RequestMapping("/entidade/v1")
17 public class FisicaResource {
18
19     @Autowired
20     private FisicaService fisicaService;
21
22     @GetMapping("/")
23     public List<Fisica> getAllFisica() {
24         return fisicaService.findAll();
25     }
26 }
```

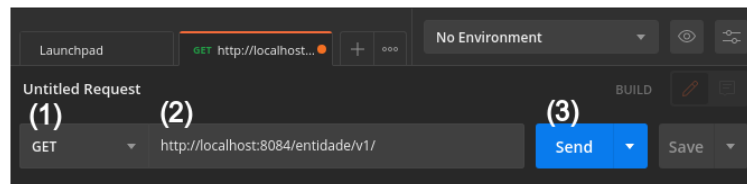
Antes de testar o comportamento de nossa API, execute a limpeza do banco de dados e faça a inserção duas entidades por meio da classe de teste `FisicaServiceTest` apresentada na Seção 2.5. A seguir, são apresentadas as linhas de comando para geração dos testes:

```
entity = InstanceGenerator.getPessoaFisica("222.333.444-55", "user1");
entity = InstanceGenerator.getPessoaFisica("222.333.444-56", "user2");
```

Diante do exposto, podemos testar nossa API utilizando a ferramenta Postman¹. Para isso, deixe o método `GET` selecionado (1), preencha o campo abaixo com a URL da API (2) e clique no botão `Send` (3) para que uma chamada seja enviada a nossa API implementada. A Figura 11 ilustra os passos de execução supracitados.

URL: `http://localhost:8084/entidade/v1/`

Figura 11 – Chamada da API via Postman: método `getAllFisica`



Fonte: Elaborada pelo autor

Na Listagem 31 é apresentado o resultado da execução referente a chamada ilustrada na Figura 11. Nessa listagem é possível observar dois objetos que foram retornados, sendo o primeiro representado pelo `idPessoa = 1` (Linha 3) e o segundo pelo `idPessoa = 6` (Linha 18). Os trechos dos objetos `acesso`, `contato` e `endereco` foram omitidos para facilitar a visualização do código.

Listagem 31 – Resultado da execução da API

```
1  [
2    {
3      "idPessoa": 1,
4      "nome": "Juca Chaves",
5      "endereco": [
6        ...
7      ],
8      "acesso": {
9        ...
10     },
11     "contato": {
12       ...
13     },
14     "cpf": "222.333.444-55",
15     "dataNascimento": "2021-07-22T19:35:18.000+00:00"
16   },
```

¹ <https://www.postman.com>

```
17    {
18        "idPessoa": 6,
19        "nome": "Juca Chaves",
20        "endereço": [
21            ...
22        ],
23        "acesso": {
24            ...
25        },
26        "contato": {
27            ...
28        },
29        "cpf": "222.333.444-55",
30        "dataNascimento": "2021-07-22T19:35:48.000+00:00"
31    }
32 ]
```

Fonte: Elaborada pelo autor

Na **Listagem 32** é apresentada a implementação do método `getFisicaByCpf`, que permite consultar uma pessoa pelo CPF. Observe que esse método foi anotado com `@GetMapping("/cpf")`, indicando que nossa API aceita uma chamada por GET com parâmetro (`cpf`). Para que um parâmetro externo seja passado ao método é necessário utilizar a anotação `@PathVariable(value = "cpf")` na assinatura (Linha 2).

Listagem 32 – Implementação do método `getFisicaByCpf`

```
1 @GetMapping("/{cpf}")
2 public Fisica getFisicaByCpf(@PathVariable(value = "cpf") String cpf) {
3     Fisica fisica = fisicaService.findByCpf(cpf);
4
5     return fisica;
6 }
```

Fonte: Elaborada pelo autor

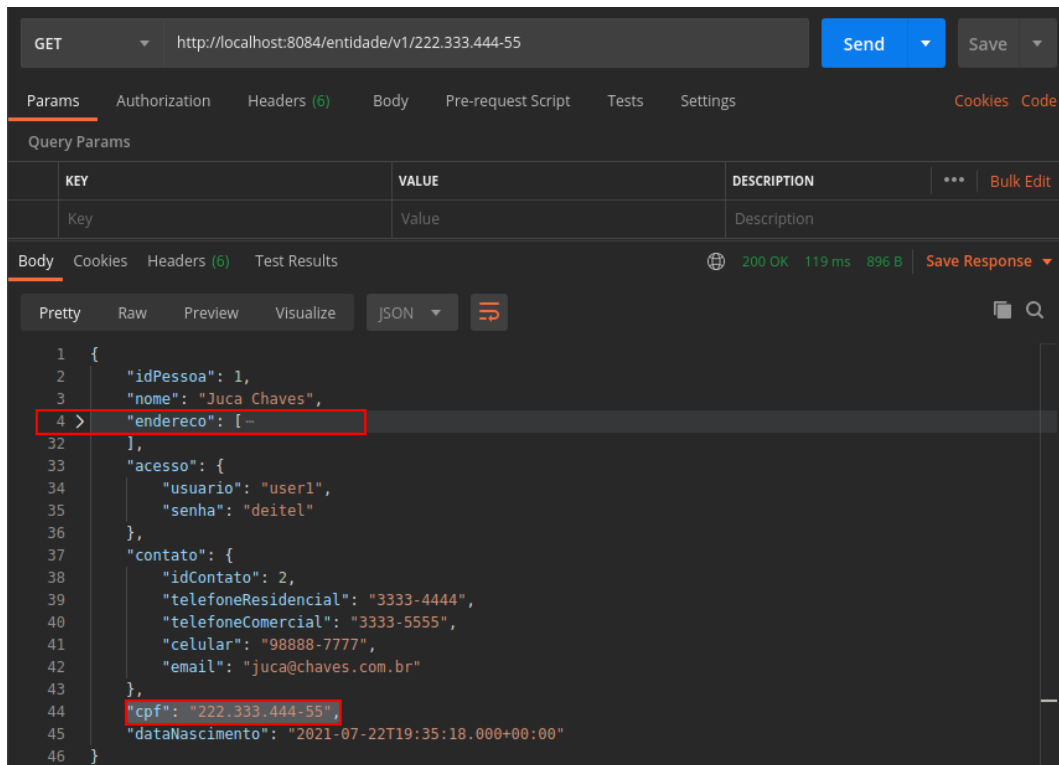
Para testar a método `getFisicaByCpf` de nossa API (**Listagem 32**), digite a seguinte URL na ferramenta Postman com a opção `GET` selecionada. A **Figura 12** ilustra o resultado da execução, onde é possível visualizar a entidade consultada (Linha 44).

URL: `http://localhost:8084/entidade/v1/222.333.444-55`

Onde (222.333.444-55) é um CPF da entidade existente no banco de dados.

Observação: os dados de endereço foram ocultos para facilitar a visualização. Veja Linha 4 destacada pela caixa em vermelho.

Agora vamos implementar um método para excluir uma pessoa pelo CPF. Esse método, chamado `delete`, é semelhante em relação ao anterior (`getFisicaByCpf`), pois possui as mesmas anotações e parâmetros. A diferença está na anotação utilizada para indicar sua finalidade

Figura 12 – Chamada da API via Postman: método getFisicaByCpf

Fonte: Elaborada pelo autor

(`@DeleteMapping("/cpf")`). Na Listagem 33 é apresentada a implementação desse método.

Listagem 33 – Implementação do método delete

```

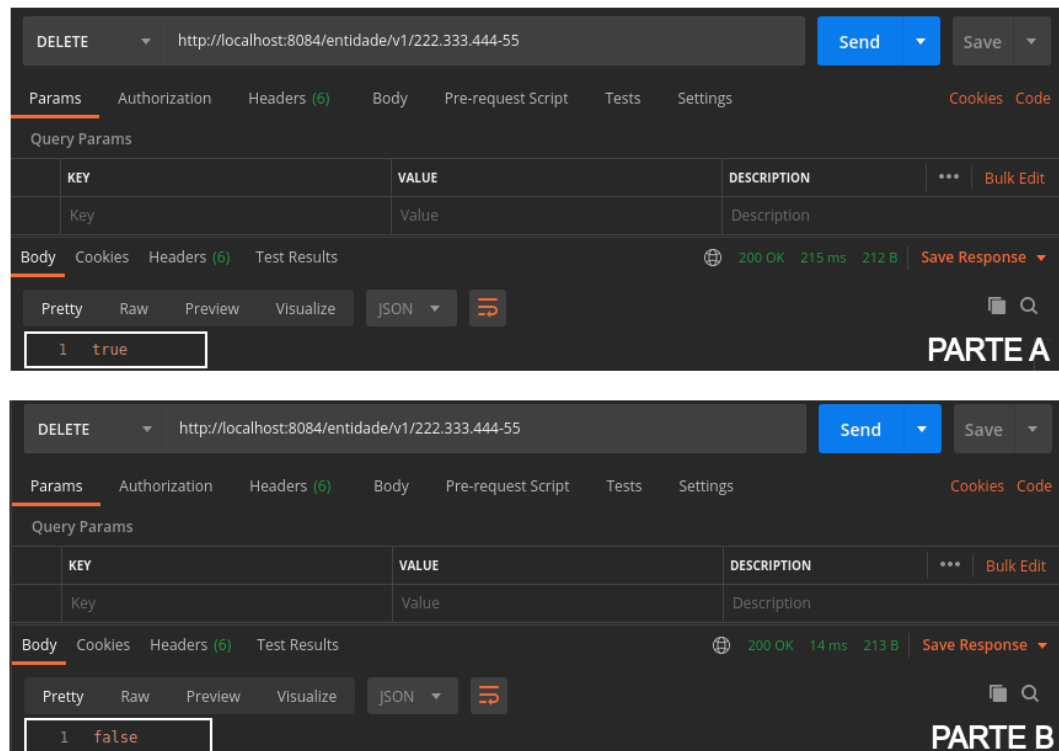
1 @DeleteMapping("/{cpf}")
2 public boolean delete(@PathVariable(value = "cpf") String cpf) {
3     boolean delete = false;
4     Fisica fisicaDelete = fisicaService.findByCpf(cpf);
5
6     if (fisicaDelete != null) {
7         fisicaService.delete(fisicaDelete);
8         delete = true;
9     }
10
11     return delete;
12 }

```

Fonte: Elaborada pelo autor

A Figura 13 ilustra a execução do método `delete` de nossa API para uma entidade existente no banco de dados (PARTE A). Observe que o método `DELETE` foi selecionado e o (`CPF = 222.333.444-55`) passado como parâmetro. De modo análogo (PARTE B), veja a execução para uma entidade inexistente no banco de dados.

URL: `http://localhost:8084/entidade/v1/222.333.444-55`

Figura 13 – Chamada da API via Postman: método delete

Fonte: Elaborada pelo autor

Até aqui nós podemos listar todas as entidades (pessoa física), consultar uma entidade pelo cpf e excluir uma pessoa física pelo cpf. Para que possamos transferir uma entidade completa do lado cliente para nossa API (lado servidor), um mapeamento da entidade deve ser realizado pela complexidade do modelo apresentado na [Figura 6](#).

3.3 Implementando a camada DTO

Nesta seção vamos implementar um método em nossa API que permite a inserção de uma pessoa física. Para isso, vamos utilizar o padrão DTO (do inglês, *Data Transfer Object*) (FOWLER *et al.*, 2002). Em síntese, esse padrão permite que as classes de nosso modelo (pacote entity) sejam “transformadas” em outras classes para facilitar a comunicação entre o lado cliente e o lado servidor de uma aplicação.

Para dar início ao desenvolvimento da camada DTO, crie um pacote de mesmo nome (`dto`) no pacote principal `br.unesp.rc.springtutorial` de nosso exemplo. Em seguida, crie uma classe chamada `EnderecoDTO` com os mesmos atributos da entidade `Endereco` do pacote `entity`. Na [Listagem 34](#) é apresentada a implementação dessa classe. Observe que a diferença entre as classe é a presença do construtor com parâmetros na referida listagem (linhas 28 a 36).

Listagem 34 – Código fonte da classe EnderecoDTO.java

```
1 package br.unesp.rc.springtutorial.dto;
```

```
2
3 import lombok.EqualsAndHashCode;
4 import lombok.Getter;
5 import lombok.Setter;
6 import lombok.ToString;
7
8 /**
9  *
10  * @author Prof. Dr. Frank J. Affonso
11  */
12 @Getter
13 @Setter
14 @ToString
15 @EqualsAndHashCode
16 public class EnderecoDTO {
17
18     private String rua;
19     private int numero;
20     private String bairro;
21     private String cep;
22     private String cidade;
23     private String estado;
24
25     public EnderecoDTO() {
26     }
27
28     public EnderecoDTO(String rua, int numero, String bairro,
29         String cep, String cidade, String estado) {
30         this.rua = rua;
31         this.numero = numero;
32         this.bairro = bairro;
33         this.cep = cep;
34         this.cidade = cidade;
35         this.estado = estado;
36     }
37 }
```

Fonte: Elaborada pelo autor

Em seguida, crie uma classe chamada `FisicaDTO` no pacote `dto`. Observe que essa classe engloba todos os atributos das classes que fazem parte de Pessoa (ou seja, `Acesso`, `Contato` e `Pessoa` e `Fisica`). Note que o atributo `dataNascimento` foi anotado com `@JsonFormat` (linha 30) para padronizar (`pattern="yyyy-MM-dd'T'HH:mm:ss.SSSZ"`) a transferência do tipo `Date`. Na [Listagem 35](#) é apresentada a implementação dessa classe.

Listagem 35 – Código fonte da classe `FisicaDTO.java`

```
1 package br.unesp.rc.springtutorial.dto;
2
3 import com.fasterxml.jackson.annotation.JsonFormat;
4 import java.util.ArrayList;
5 import java.util.Date;
6 import java.util.List;
7 import lombok.EqualsAndHashCode;
8 import lombok.Getter;
9 import lombok.Setter;
10 import lombok.ToString;
11
12 /**
13  *
14  * @author Prof. Dr. Frank J. Affonso
15  */
16 @Getter
17 @Setter
18 @ToString
19 @EqualsAndHashCode
20 public class FisicaDTO {
21
22     private String usuario;
23     private String senha;
24     private String telefoneResidencial;
25     private String telefoneComercial;
26     private String celular;
27     private String email;
28     private String nome;
29     private String cpf;
30     @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss.SSSZ")
31     private Date dataNascimento;
32     private List<EnderecoDTO> endereco;
33
34     public FisicaDTO() {
35         this.endereco = new ArrayList<>();
36     }
37
38     public FisicaDTO(String usuario, String senha, String
39         ↳ telefoneResidencial,
40         String telefoneComercial, String celular, String email, String
41         ↳ nome,
42         String cpf, Date dataNascimento, List<EnderecoDTO> endereco) {
43         this.usuario = usuario;
44         this.senha = senha;
```

```
43     this.telefoneResidencial = telefoneResidencial;
44     this.telefoneComercial = telefoneComercial;
45     this.celular = celular;
46     this.email = email;
47     this.nome = nome;
48     this.cpf = cpf;
49     this.dataNascimento = dataNascimento;
50     this.endereco = endereco;
51 }
52 }
```

Fonte: Elaborada pelo autor

O próximo passo é criar a classe responsável por “traduzir” os objetos de transição para nosso modelo. O processo contrário não será apresentado neste documento por motivo de escopo. Para isso, crie um pacote chamado `assembler` em `dto`. Em seguida, crie uma classe chamada `FisicaAssembler` e implemente o método `dtoToEntityModel`, que recebe um objeto do tipo `FisicaDTO` e retorna um objeto do tipo `Fisica`. Na [Listagem 36](#) é apresentada a implementação da classe `FisicaAssembler.java`.

Listagem 36 – Código fonte da classe `FisicaAssembler.java`

```
1 package br.unesp.rc.springtutorial.dto.assembler;
2
3 import br.unesp.rc.springtutorial.dto.EnderecoDTO;
4 import br.unesp.rc.springtutorial.dto.FisicaDTO;
5 import br.unesp.rc.springtutorial.entity.Acesso;
6 import br.unesp.rc.springtutorial.entity.Contato;
7 import br.unesp.rc.springtutorial.entity.Endereco;
8 import br.unesp.rc.springtutorial.entity.Fisica;
9
10 /**
11  *
12  * @author Prof. Dr. Frank J. Affonso
13  */
14 public class FisicaAssembler {
15
16     private FisicaAssembler() {
17
18     }
19
20     public static Fisica dtoToEntityModel(FisicaDTO dto) {
21         Fisica fisica = new Fisica();
22
23         fisica.setCpf(dto.getCpf());
24         fisica.setDataNascimento(dto.getDataNascimento());
```

```
25     fisica.setNome(dto.getNome());
26
27     Acesso a = new Acesso();
28     a.setUsuario(dto.getUsuario());
29     a.setSenha(dto.getSenha());
30     fisica.setAcesso(a);
31
32     Contato c = new Contato();
33     c.setTelefoneResidencial(dto.getTelefoneResidencial());
34     c.setTelefoneComercial(dto.getTelefoneComercial());
35     c.setCelular(dto.getCelular());
36     c.setEmail(dto.getEmail());
37     fisica.setContato(c);
38
39     for (EnderecoDTO edto : dto.getEndereco()) {
40         Endereco e = new Endereco();
41         e.setRua(edto.getRua());
42         e.setNumero(edto.getNumero());
43         e.setBairro(edto.getBairro());
44         e.setCep(edto.getCep());
45         e.setCidade(edto.getCidade());
46         e.setEstado(edto.getEstado());
47         fisica.setEndereco(e);
48     }
49
50     return fisica;
51 }
52 }
```

Fonte: Elaborada pelo autor

3.4 Implementando a camada resource (continuação...)

Após implementar as classes de transição, podemos iniciar a implementação do método `save` em nossa API (classe `FisicaResource`). Na **Listagem 37** é apresentada a implementação do método `save`.

Listagem 37 – Código fonte do método `save`

```
1 @PostMapping("/")
2 public boolean saveFisica(@RequestBody FisicaDTO fisicaDto) {
3     boolean insert = false;
4
5     Fisica fisica = FisicaAssembler.dtoToEntityModel(fisicaDto);
6     Fisica fisicaInsert = fisicaService.save(fisica);
```

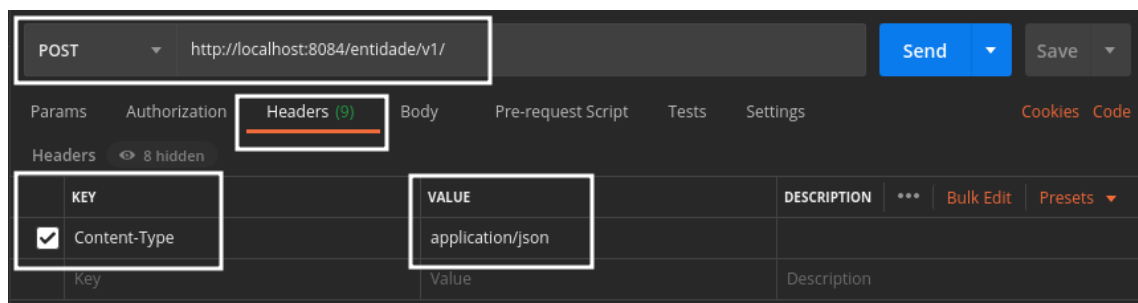
```
7     if (fisicaInsert != null) {
8         insert = true;
9     }
10
11     return insert;
12 }
```

Fonte: Elaborada pelo autor

Observe que esse método (`save`) foi anotado com `@PostMapping("/")` (Linha 1), que indica o método HTTP que será utilizado (`POST`), assim como sua rota de chamada (`/`). Note também que esse método recebe por parâmetro um objeto (`fisicaDto`) da classe `FisicaDTO`, o qual foi anotado com `@RequestBody` (Linha 2). Em síntese, o Spring desserializa automaticamente o documento JSON para um tipo Java, assumindo que um documento apropriado seja especificado. Dessa forma, o parâmetro que anotamos com `@RequestBody` deve corresponder ao JSON enviado do nosso cliente (ferramenta Postman). Para executar uma chamada do método (`save`), algumas configurações na ferramenta Postman devem ser realizadas, a saber:

1. Preencher a **URL**: `http://localhost:8084/entidade/v1/`
2. Selecionar o método `POST`
3. Na aba `Headers`, setar `content-Type` e `application/json` nos campos `KEY` e `VALUE`, como ilustra [Figura 14](#).

Figura 14 – Chamada da API via Postman: método save - Configuração



Fonte: Elaborada pelo autor

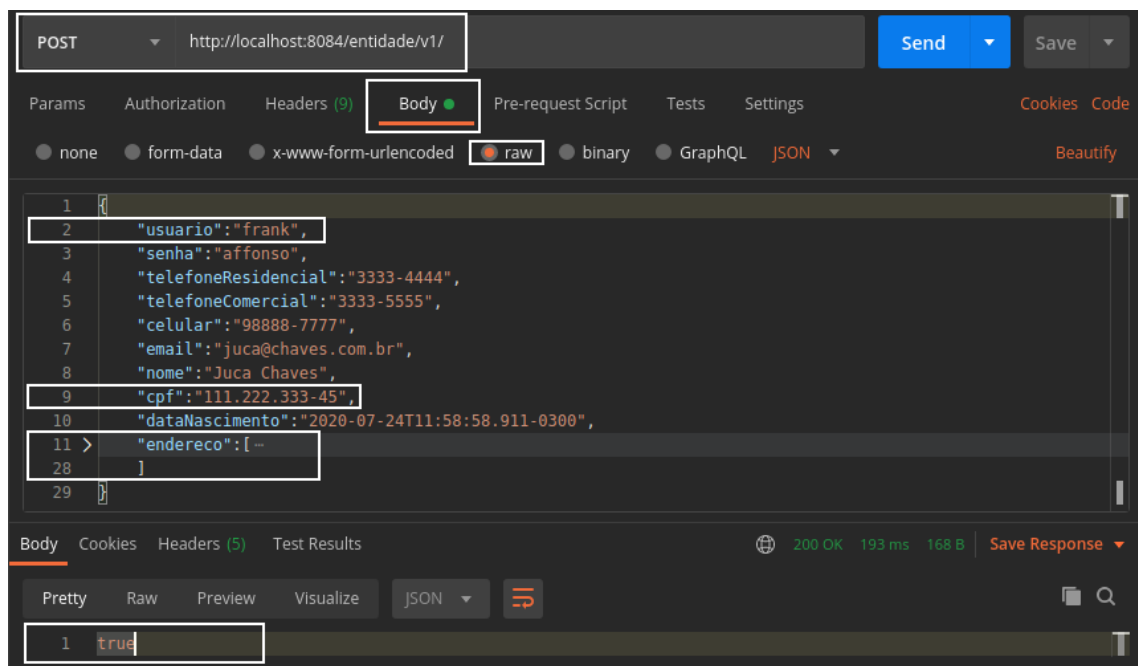
4. Na aba `Body`, selecione a opção `raw`, insira o código JSON referente ao objeto a ser enviado ([Listagem 38](#)) e clique em `Send`. Vale ressaltar que o código JSON exibido nessa listagem é para um objeto do tipo `FisicaDTO`. Os campos das linhas 2 (`usuario`) e 9 (`cpf`) devem ser alterados a cada execução, pois representam atributos de unicidade no banco de dados. A [Figura 15](#) ilustra a configuração supracitada, assim como o resultado da execução. Os dados de endereço (Linha 11) foram ocultos para facilitar a visualização.

Listagem 38 – Código fonte de um objeto do tipo FisicaDTO

```
1 {
2   "usuario":"frank",
3   "senha":"affonso",
4   "telefoneResidencial":"3333-4444",
5   "telefoneComercial":"3333-5555",
6   "celular":"98888-7777",
7   "email":"juca@chaves.com.br",
8   "nome":"Juca Chaves",
9   "cpf":"111.222.333-45",
10  "dataNascimento":"2020-07-24T11:58:58.911-0300",
11  "endereco":[
12    {
13      "rua":"Avenida 24A",
14      "numero":1515,
15      "bairro":"Bela Vista",
16      "cep":"13506-900",
17      "cidade":"Rio Claro",
18      "estado":"SP"
19    },
20    // ATENÇÃO: Para inserir mais de uma entidade de endereço. Duplique o
    ↳ trecho acima e altere apenas o número.
21  ]
22 }
```

Fonte: Elaborada pelo autor

Figura 15 – Chamada da API via Postman: método save - Resultado



Fonte: Elaborada pelo autor

Após implementar a inserção de uma entidade, também é importante que a mesma possa

ser atualizada. Nesse sentido dois métodos HTTP podem ser utilizados, a saber:

- **PUT**, que permite atualizar um recurso existente por completo. Por exemplo, a API permite atualizar todos os campos da entidade no banco de dados; e
- **PATCH**, que permite atualizar parcialmente um recurso existente. Por exemplo, somente parte dos dados de uma entidade são atualizados no banco de dados e os demais permanecem inalterados.

Para dar continuidade ao desenvolvimento de nosso exemplo, somente o método **PUT** será considerado. Dessa forma, apenas um método que faz a atualização de uma entidade pessoa física será implementado em nossa API. Vale destacar que esse método é relativamente semelhante ao método **save**, já detalhado nesta seção. Basicamente, a atualização de uma entidade é composta por cinco passos, a saber:

1. Transferência de um objeto DTO para o lado servidor;
2. Conversão desse objeto para um objeto de modelo de entidades;
3. Localização desse objeto no banco de dados;
4. Mapeamento desse objeto (existente no banco de dados) para o novo objeto (DTO); e
5. Persistência do novo objeto;

Os Passos 1 a 3 já foram apresentados neste documento, sendo necessário detalhar apenas a implementação dos dois últimos. Na **Listagem 39** é apresentada o trecho de código do método **update**. Observe na Linha 1 a anotação **@PutMapping("/")** (Linha 1), que indica o tipo de chamada e a rota que o método irá mesmo responder. Assim, como o método **save**, este também recebe, via parâmetro, um objeto do tipo **FisicaDTO**. Detalhes dessa anotação não serão apresentados aqui para evitar redundância de informação.

Listagem 39 – Código fonte do método update

```
1 @PutMapping("/")
2 public boolean update(@RequestBody FisicaDTO fisicaDto) {
3     boolean update = false;
4
5     Fisica newFisica = FisicaAssembler.dtoToEntityModel(fisicaDto);
6     Fisica fisicaUpdate = fisicaService.findByCpf(newFisica.getCpf());
7
8     FisicaMapper.update(fisicaUpdate, newFisica);
9
10    Fisica fisicaUpdated = fisicaService.update(fisicaUpdate);
```

```
11     if (fisicaUpdated != null) {
12         update = true;
13     }
14
15     return update;
16 }
```

Fonte: Elaborada pelo autor

Na Linha 5 ocorre a conversão do objeto DTO para um objeto do modelo (`Fisica`) e um objeto `newFisica` é criado. Já na Linha 6, uma entidade é recuperada e um objeto `fisicaUpdate` é criado. O motivo dessas operações é que o objeto recuperado do banco de dados contém todas as chaves (identificadores) gerados quando um objeto é persistido. Para realizar uma operação de atualização, essas chaves devem ser preservadas. Na Linha 8 ocorre o mapeamento dos objetos, onde os dados do objeto `newFisica` são transferidos para `fisicaUpdate`. Por fim, na Linha 10 ocorre a atualização do objeto e a `flag` é atualizada para `true` se a operação foi realizada com sucesso.

Para que a operação de atualização seja realizada, uma classe chamada `FisicaMapper` deve ser implementada. Para isso, crie um pacote `mapper` em `br.unesp.rc.springtutorial.entity`. Na **Listagem 40** é apresentada o código fonte da classe `FisicaMapper.java`. Basicamente, essa classe possui apenas um método chamado `update` (Linha 17) para transferir os dados de `newFisica` para `fisicaUpdate`, ambos objetos da classe `Fisica`.

Listagem 40 – Código fonte da classe FisicaMapper

```
1 package br.unesp.rc.springtutorial.entity.mapper;
2
3 import br.unesp.rc.springtutorial.entity.Acesso;
4 import br.unesp.rc.springtutorial.entity.Contato;
5 import br.unesp.rc.springtutorial.entity.Endereco;
6 import br.unesp.rc.springtutorial.entity.Fisica;
7
8 /**
9  *
10  * @author Prof. Dr. Frank J. Affonso
11  */
12 public class FisicaMapper {
13
14     private FisicaMapper() {
15     }
16
17     public static void update(Fisica fisicaUpdate, Fisica newFisica) {
18
19         fisicaUpdate.setDataNascimento(newFisica.getDataNascimento());
20         fisicaUpdate.setNome(newFisica.getNome());
```

```
21
22     Acesso a = fisicaUpdate.getAcesso();
23     a.setUsuario(newFisica.getAcesso().getUsuario());
24     a.setSenha(newFisica.getAcesso().getSenha());
25     fisicaUpdate.setAcesso(a);
26
27     Contato c = fisicaUpdate.getContato();
28     c.setTelefoneResidencial(newFisica.getContato().getTelefoneResidencial());
29     c.setTelefoneComercial(newFisica.getContato().getTelefoneComercial());
30     c.setCelular(newFisica.getContato().getCelular());
31     c.setEmail(newFisica.getContato().getEmail());
32     fisicaUpdate.setContato(c);
33
34     int i = 0;
35     for (Endereco e : fisicaUpdate.getEndereco()){
36         Endereco edto = newFisica.getEndereco().get(i);
37         e.setRua(edto.getRua());
38         e.setNumero(edto.getNumero());
39         e.setBairro(edto.getBairro());
40         e.setCep(edto.getCep());
41         e.setCidade(edto.getCidade());
42         e.setEstado(edto.getEstado());
43         i++;
44     }
45 }
46 }
```

Fonte: Elaborada pelo autor

A execução de uma atualização de entidade segue o mesmo procedimento da inserção. Por esse motivo, alguns passos não serão reportados novamente aqui para evitar redundância de informações. Na **Listagem 41** é apresentado o código fonte em JSON para uma entidade a ser atualizada. Observe que esse código é relativamente similar ao apresentado na **Listagem 38**, sendo alguns campos modificados em valores ou acrescido a palavra *UPDATED*.

Listagem 41 – Código fonte de um objeto do tipo FisicaDTO para atualização

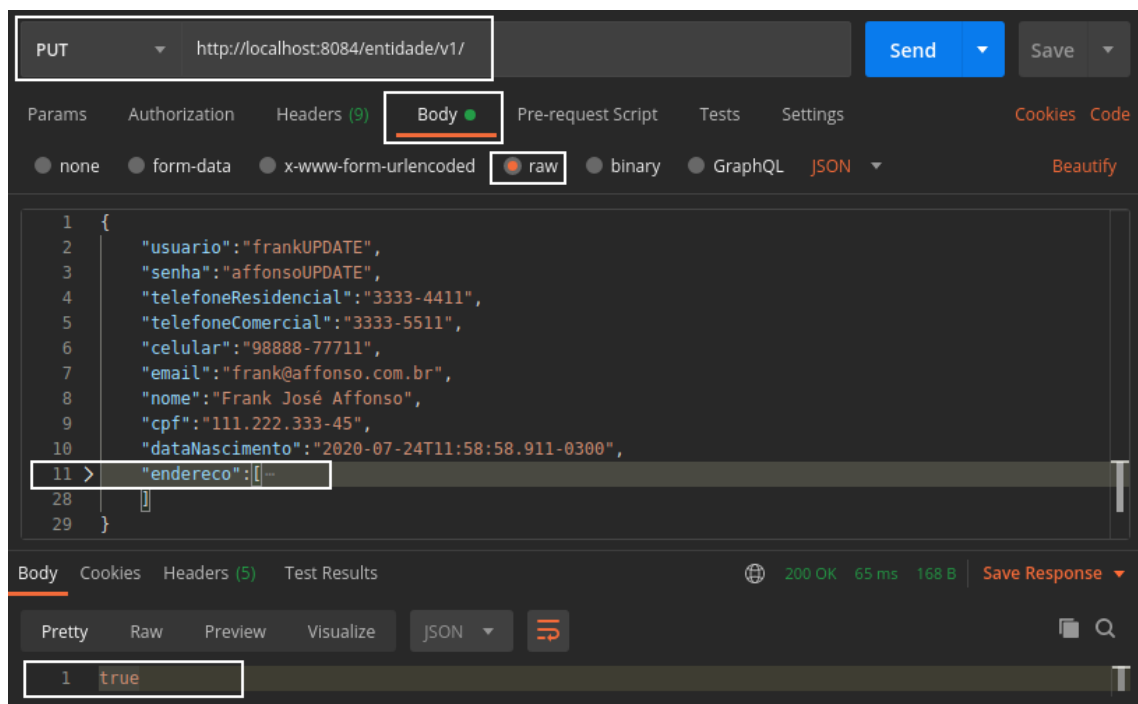
```
1 {
2     "usuario":"frankUPDATE",
3     "senha":"affonsoUPDATE",
4     "telefoneResidencial":"3333-4411",
5     "telefoneComercial":"3333-5511",
6     "celular":"98888-77711",
7     "email":"frank@affonso.com.br",
8     "nome":"Frank Jose Affonso",
9     "cpf":"111.222.333-45",
10    "dataNascimento":"2020-07-24T11:58:58.911-0300",
```

```
11     "endereco": [  
12         {  
13             "rua": "Avenida 24A UPDATE",  
14             "numero": 1515,  
15             "bairro": "Bela Vista UPDATE",  
16             "cep": "13506-900",  
17             "cidade": "Rio Claro",  
18             "estado": "SP"  
19         },  
20         {  
21             "rua": "Avenida 24A UPDATE",  
22             "numero": 2020,  
23             "bairro": "Bela Vista",  
24             "cep": "13506-900",  
25             "cidade": "Rio Claro",  
26             "estado": "SP"  
27         }  
28     ]  
29 }
```

Fonte: Elaborada pelo autor

A **Figura 16** ilustra a configuração para atualização de uma entidade com base na **Listagem 38**, assim como o resultado da execução. Os dados de endereço (Linha 11) foram ocultos para facilitar a visualização.

Figura 16 – Chamada da API via Postman: método update



Fonte: Elaborada pelo autor

3.5 Considerações finais

O conteúdo apresentado neste documento permite aos interessados iniciar o desenvolvimento de APIs do tipo RESTful, onde foi implementada as seguintes funcionalidades para a entidade pessoa física:

- Consulta pelo CPF;
- Listagem de todas entidades;
- Exclusão de uma entidade pelo CPF;
- Inserção de uma entidade; e
- Atualização de uma entidade.

O documento foi elaborado com base em padrões e boas práticas de Engenharia de Software. Dessa forma, acredita-se que os interessados possam aplicar o conhecimento apresentados em projeto de software de maior escala.

4 Utilizando OpenAPI 3 e Swagger para documentação de APIs REST

Autor(es):

Prof. Dr. Frank J. Affonso

Pré-requisitos:

Java: OpenJDK 64-Bit Server VM AdoptOpenJDK-11.0.11+9

Spring Boot: versão 2.5.2

Netbeans IDE: versão 12.3

MySQL Server: versão 8.0.26 *Community Server* - GPL

MySQL Workbench: versão 8.0.26 *Community*

Sistema Operacional: Linux - Kubuntu 20.04.1

4.1 Considerações Iniciais

Antes de iniciar a apresentação desta seção, uma importante consideração deve ser destacada. Para que os interessados executem os passos aqui apresentados para documentação de APIs REST, o desenvolvimento da aplicação reportado no [Capítulo 3](#) deve ser realizado. Assim como no capítulo supracitado, o desenvolvimento do exemplo será realizado em continuidade, sendo reportados aqui apenas os passos para adaptação da codificação existente e/ou criação de novas funcionalidades.

4.2 Visão geral

Antes de iniciar a apresentação dos passos para a documentação de APIs REST, duas considerações importantes precisam ser definidas. A primeira é que esta etapa é essencial para o desenvolvimento desse tipo de API ([ERL, 2005](#); [ERL, 2007](#); [ERL et al., 2017](#)). A segunda está relacionada ao correto entendimento sobre os termos OpenAPI e Swagger. De acordo com [Swagger \(2017\)](#), o primeiro termo é o nome oficial de uma especificação e o segundo é o

nome de ferramentas que implementam tal especificação para apoiar a atividade de documentação de APIs. Maiores detalhes sobre esses termos podem ser obtidos na referência supracitada.

4.3 Configurando o springdoc-openapi

De acordo com [Springdoc-openapi \(2021\)](#), [springdoc-openapi](#) é uma biblioteca Java que permite automatizar a geração de documentação em APIs desenvolvidas em Spring Boot. Para isso, apenas uma dependência ([Listagem 42](#)) deve ser adicionada ao arquivo [pom.xml](#) da aplicação, sem que nenhuma configuração extra seja necessária para visualizar o resultado.

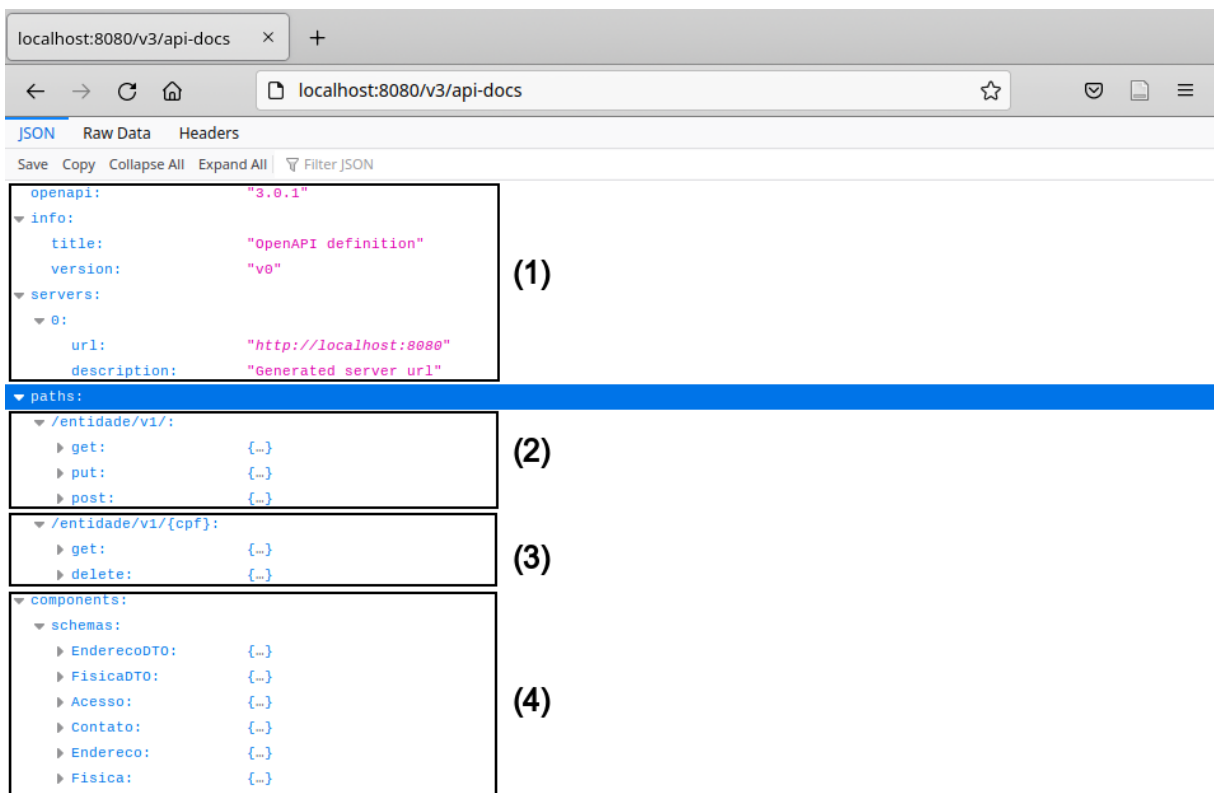
Listagem 42 – Dependência do springdoc-openapi-ui

```
1 <dependency>
2   <groupId>org.springdoc</groupId>
3   <artifactId>springdoc-openapi-ui</artifactId>
4   <version>1.5.9</version>
5 </dependency>
```

Fonte: Elaborada pelo autor

Diante do exposto, acesse a URL (<http://localhost:8080/v3/api-docs>) no navegador de sua preferência e veja o resultado ilustrado na [Figura 17](#).

Figura 17 – Chamada da API via open-docs

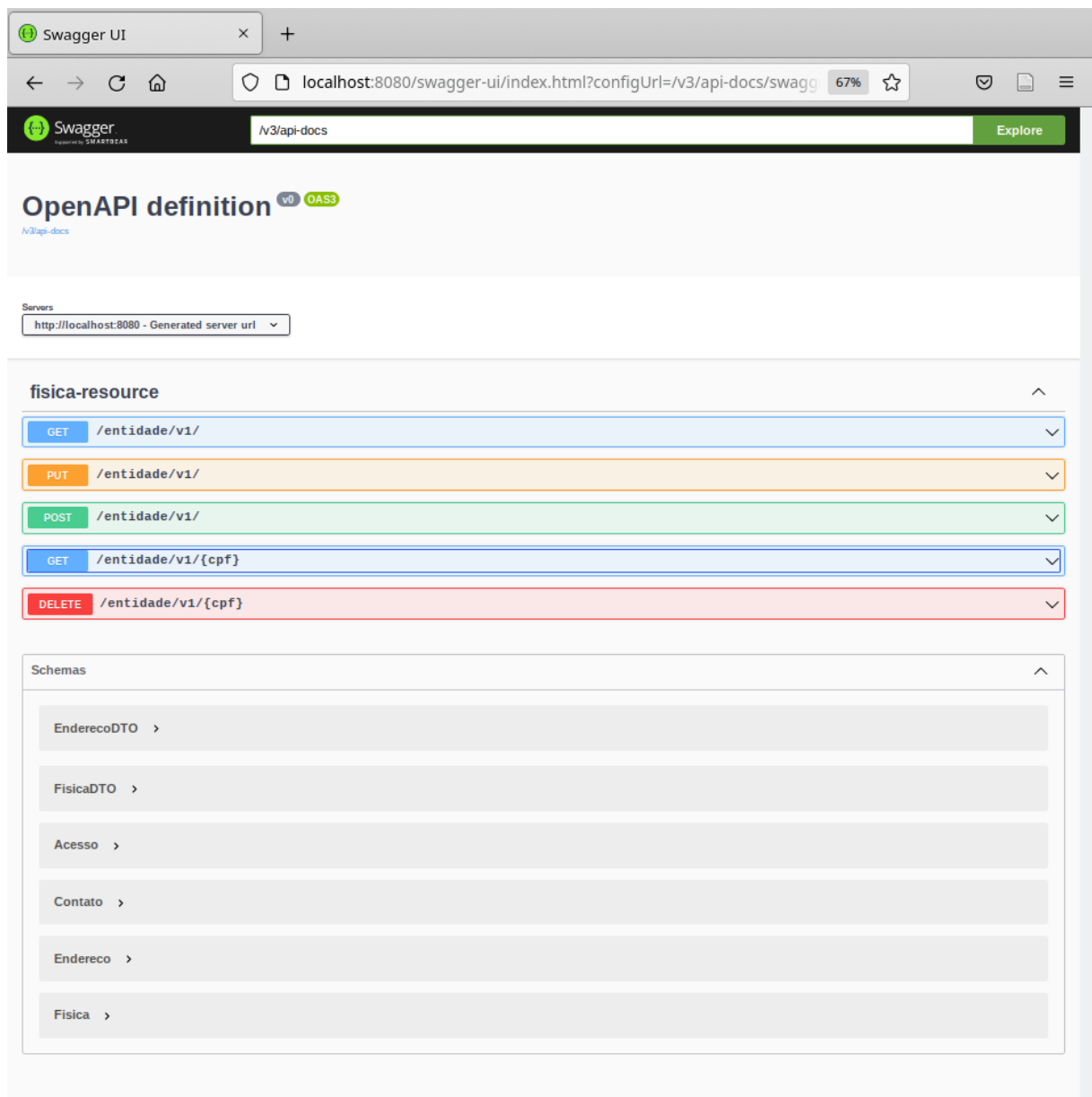


Fonte: Elaborada pelo autor

Em (1) pode-se observar o cabeçalho de nossa API, onde a maioria das informações não foram configuradas, a saber: *title*, *version* e *description*. Em (2) e (3) é possível observar as APIs implementadas, sendo organizadas pela rota de resposta (`"/entidade/v1/"` e `/entidade/v1/cpf`). Por fim, em (4) são exibidos os tipos de dados trocados/transferidos de/para as APIs.

Em seguida, acesse a URL (`http://localhost:8080/swagger-ui.html`) no navegador de sua preferência e veja o resultado ilustrado na **Figura 18**.

Figura 18 – Chamada da API via Swagger

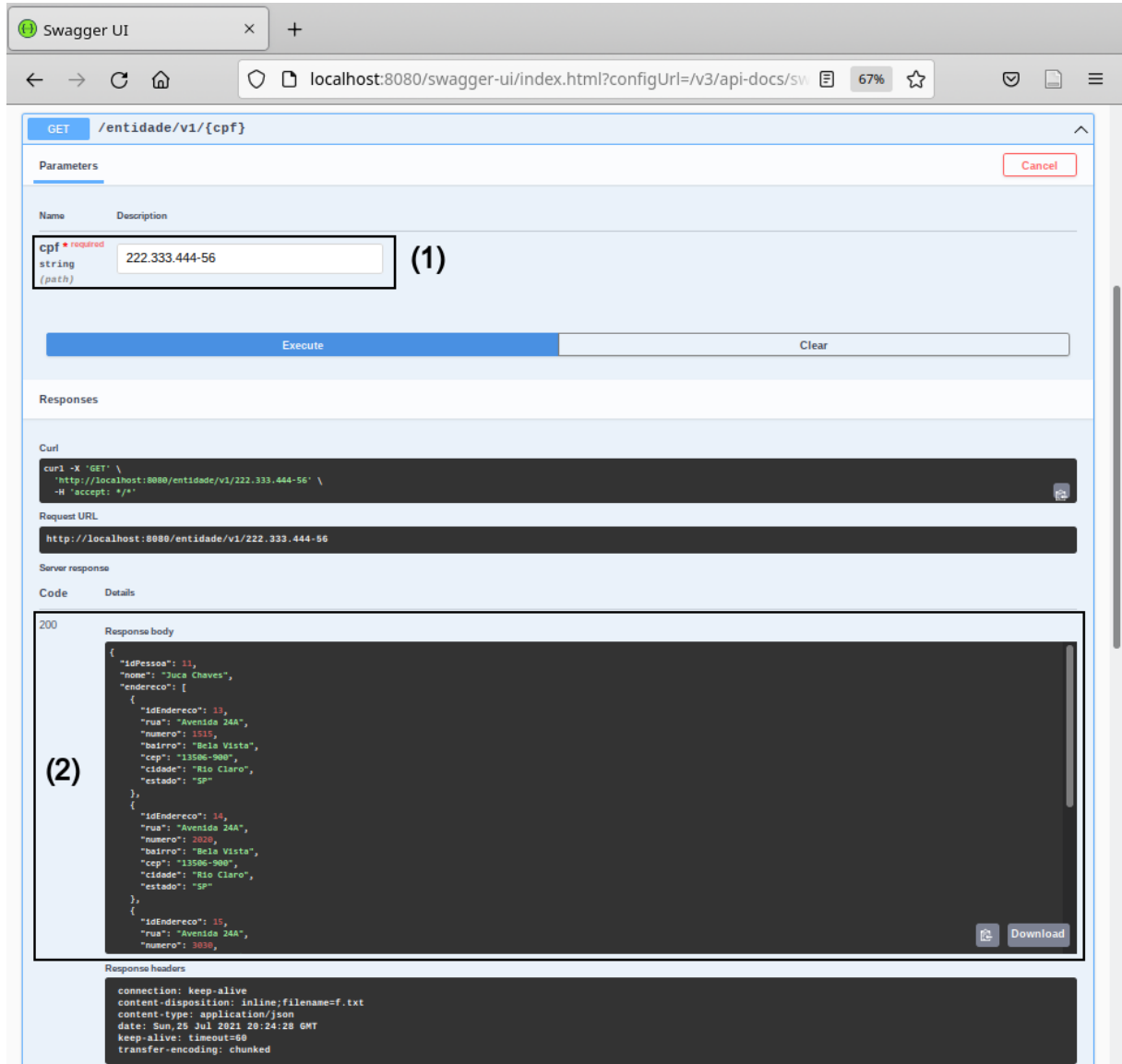


Fonte: Elaborada pelo autor

Basicamente, em `fisca-resource` é possível visualizar os métodos implementados em nossa API, assim como suas rotas de acesso. Em `schemas` é possível ver os objetos que são utilizados pelos métodos de nossa API. Além disso, esse *front-end* permite que o desenvolvedor

possa experimentar o funcionamento dos métodos disponibilizados. Para isso, a funcionalidade para localizar uma entidade pelo cpf será considerada, como ilustra a **Figura 19**

Figura 19 – Consulta de uma entidade pelo CPF via Swagger



Fonte: Elaborada pelo autor

Em (1) o interessado/desenvolvedor digita o CPF a ser consultado e, caso a entidade exista no banco de dados, a mesma é retornada como ilustrado em (2). Por motivos de escopo, outras operações não serão apresentadas nesta seção. No entanto, pode-se dizer que o Swagger-UI oferece um suporte importante para documentação e execução de APIs REST.

Visando facilitar o acesso das API, o `springdoc-openapi` permite customizar as URLs de acesso e apresentação dos métodos da API. Para isso, o arquivo `application.properties` deve ser modificado. Nesse sentido adicione as seguintes entradas:

- `springdoc.api-docs.path=/api-docs`, que irá modificar o acesso da URL apresentada na

Figura 17.

DE.: `http://localhost:8080/v3/api-docs/`

PARA: `http://localhost:8080/api-docs`

- `springdoc.swagger-ui.path=/swagger-ui-entity.html`, de modo análogo ao anterior, a URL apresentada na Figura 18 também será alterada.

DE.: `http://localhost:8080/swagger-ui.html`

PARA: `http://localhost:8080/swagger-ui-entity.html`

- `springdoc.swagger-ui.operationsSorter=method`, que irá ordenar a apresentação dos métodos de nossa API. Após inserir essa entrada, atualize a página da API e compare o resultado com a Figura 18.

Visando facilitar o acesso aos endereços supracitados, uma página chamada `index.html` será criada em `Other Sources` -> `static`. Na Listagem 43 é apresentado o código fonte da página `index.html`.

Listagem 43 – Código fonte da página index

```

1 <!DOCTYPE html>
2 <!--
3 To change this license header, choose License Headers in Project
   ↪ Properties.
4 To change this template file, choose Tools | Templates
5 and open the template in the editor.
6 -->
7 <html>
8   <head>
9     <title>TODO supply a title</title>
10    <meta charset="UTF-8">
11    <meta name="viewport" content="width=device-width,
   ↪ initial-scale=1.0">
12  </head>
13  <body>
14    <div>
15      <h1> Entity Application is up!</h1>
16      <h3> Springdoc-openapi v1.5.9</h3>
17    </div>
18    <div>
19      <b>
20        API documentation:
21      </b>
22      <a href="http://localhost:8080/api-docs">Api-docs</a>
23    </div>

```

```

24     <br>
25     <div>
26         <b>
27             Swagger UI:
28         </b>
29         <a href="http://localhost:8080/swagger-ui-entity.html">
30             ↪ Swagger-UI </a>
31     </div>
32 </body>
33 </html>

```

Fonte: Elaborada pelo autor

4.4 Documentando nosso modelo

A biblioteca `springdoc-openapi` permite também documentar modelos de entidades que utilizam anotações de validação **JSR-303 Bean**, tais como: `@NotNull`, `@NotBlank`, `@Size`, `@Min` e `@Max`. Em resumo, a biblioteca utiliza essas anotações para gerar documentação de esquema adicional para as restrições correspondentes. Para isso, vamos considerar duas classes/entidades de nosso modelo: `Fisica` e `Pessoa`.

Inicialmente, a entidade `Fisica` será anotada com `@NotBlank` e `@Size(min = 11, max = 14)` para que um `cpf` não seja enviado “em branco” e que tenha tamanho entre 11 e 14. A [Figura 20](#) ilustra a situação da API para as referidas classes antes e depois da inserção das anotações. Os quadro em vermelho destacam a documentação gerada em função da inserção das anotações supracitadas.

Figura 20 – Documentando a API



Fonte: Elaborada pelo autor

Observe que o conteúdo exposto na [Figura 20](#) foi apenas para ilustrar o uso das anotações de validação com a biblioteca. Por motivos de escopo, as demais entidades não foram anotadas,

cabendo ao projetista da API definir essas informações quando uma API for desenvolvida.

Identificar os códigos HTTP em uma API REST também é parte da documentação. Para isso, vamos criar um pacote chamado `exception` em `br.unesp.rc.springtutorial.resource`. Em seguida, crie uma classe chamada `GlobalControllerExceptionHandler` para lidar com os referidos códigos, como apresentado na [Listagem 44](#). A seguir, uma breve descrição de cada anotação é apresentada com base na documentação oficial ([SPRINGAPI, 2021](#)).

Listagem 44 – Código fonte da classe `GlobalControllerExceptionHandler`

```
1 package br.unesp.rc.springtutorial.resource.exception;
2
3 import org.springframework.core.convert.ConversionFailedException;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.ExceptionHandler;
7 import org.springframework.web.bind.annotation.ResponseStatus;
8 import org.springframework.web.bind.annotation.RestControllerAdvice;
9
10 /**
11  *
12  * @author Prof. Dr. Frank J. Affonso
13  */
14 @RestControllerAdvice
15 public class GlobalControllerExceptionHandler {
16
17     @ExceptionHandler(ConversionFailedException.class)
18     @ResponseStatus(HttpStatus.BAD_REQUEST)
19     public ResponseEntity<String> handleConnversion(RuntimeException ex) {
20         return new ResponseEntity<>(ex.getMessage(),
21             ↳ HttpStatus.BAD_REQUEST);
22     }
23
24     @ExceptionHandler(EntityNotFoundException.class)
25     @ResponseStatus(HttpStatus.NOT_FOUND)
26     public ResponseEntity<String> handleEntityNotFound(RuntimeException
27         ↳ ex) {
28         return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
29     }
30 }
```

Fonte: Elaborada pelo autor

- `@RestControllerAdvice` (Linha 14), permite que classe “aconselhadoras” sejam criadas para que erros (`@ExceptionHandler`) e respostas HTTPs sejam obtidas (`@ResponseBody`) e

manipuladas de maneira adequada.

- `@ExceptionHandler` (Linhas 17 e 21) é uma anotação que permite lidar com exceções em classes e/ou métodos manipuladores. O parâmetro `ConversionFailedException.class` (Linha 17) é exceção a ser lançada quando uma tentativa de conversão de tipo real falhar. Já o parâmetro `EntityNotFoundException.class` (Linha 21) é uma exceção customizada para entidade que se pretende manipular. Para isso, crie uma classe com o mesmo nome no pacote `exception` e insira o código apresentado na [Listagem 45](#). Observe que essa classe estende `RuntimeException` e sobrescreve o método `getMessage`.

Listagem 45 – Código fonte da classe `EntityNotFoundException`

```
1 package br.unesp.rc.springtutorial.resource.exception;
2
3 /**
4  *
5  * @author Prof. Dr. Frank J. Affonso
6  */
7 public class EntityNotFoundException extends RuntimeException {
8
9     public EntityNotFoundException() {
10     }
11
12     @Override
13     public String getMessage() {
14         return super.getMessage();
15     }
16
17 }
```

Fonte: Elaborada pelo autor

- `@ResponseStatus` (Linhas 18 e 24) é uma anotação que serve para sinalizar para um método ou uma classe de exceção o código de status e o motivo que devem ser retornados. `HttpStatus` é um `enum` para os códigos de status HTTP, onde `BAD_REQUEST` é o código 400 (*Bad request*) e `NOT_FOUND` é o código 404 (*Not found*). Como resultado, agora podemos ver a documentação dos códigos de resposta 400 e 404, conforme ilustra a [Figura 21](#).

Para finalizar a documentação da nossa API, vamos adicionar informações aos métodos ou *endpoints* de nossa API. Para mostrar a aplicabilidade desse tipo de documentação, duas anotações serão utilizadas (`@Operation` e `@ApiResponse`). Para efeitos demonstrativos apenas o método `getFisicaByCpf`, que responde pelo *endpoint* `"/cpf"` será utilizado. Na [Listagem 46](#) é apresentado o código fonte já com as anotações de documentação do método `getFisicaByCpf`.

Figura 21 – Códigos HTTP

The image shows the 'Responses' section of a Swagger UI. It lists three HTTP status codes with their descriptions and associated details:

Code	Description
200	OK
400	Bad Request
404	Not Found

For each status code, there is a 'Media type' dropdown menu set to '*/*' and an 'Example Value' field. The example values are 'true' for 200, 'string' for 400, and 'string' for 404.

Fonte: Elaborada pelo autor

Listagem 46 – Código fonte da documentação do método getFisicaByCpf

```
1 // ... código omitido
2
3 @RestController
4 @RequestMapping("/entidade/v1")
5 public class FisicaResource {
6
7     // ... código omitido
8 }
```

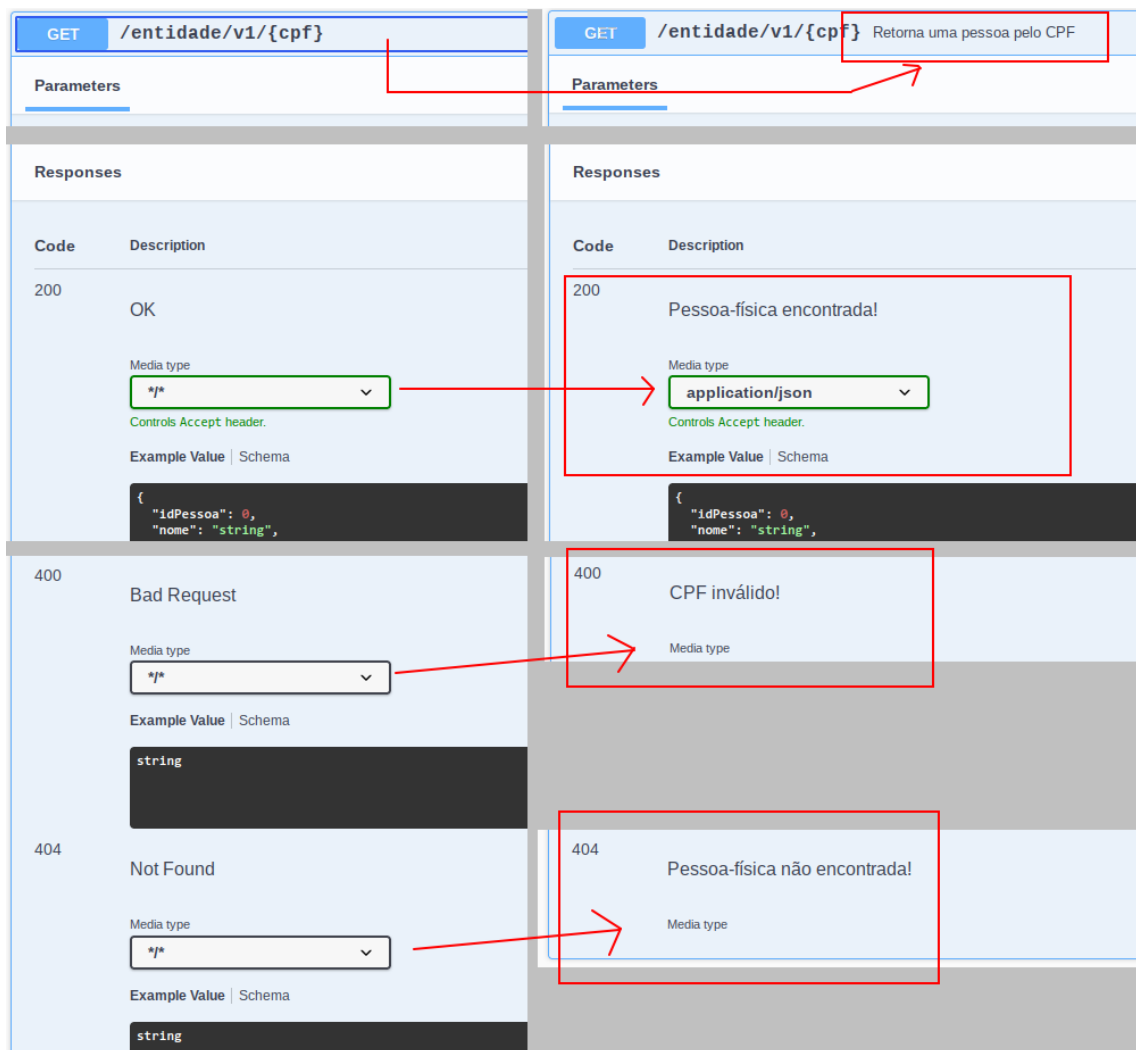
```
9      @Operation(summary = "Retorna uma pessoa pelo CPF")
10      @ApiResponses(
11          value = {
12              @ApiResponse(
13                  responseCode = "200",
14                  description = "Pessoa-física encontrada!",
15                  content = {
16                      @Content(
17                          mediaType = "application/json",
18                          schema = @Schema(implementation =
19                              ↪ Fisica.class)
20                      )
21              },
22              @ApiResponse(
23                  responseCode = "400",
24                  description = "CPF inválido!",
25                  content = @Content
26              ),
27              @ApiResponse(
28                  responseCode = "404",
29                  description = "Pessoa-física não encontrada!",
30                  content = @Content
31              )
32          }
33      )
34      @GetMapping("/{cpf}")
35      public Fisica getFisicaByCpf(@PathVariable(value = "cpf") String cpf) {
36          Fisica fisica = fisicaService.findByCpf(cpf);
37
38          return fisica;
39      }
40
41      // ... código omitido
42  }
```

Fonte: Elaborada pelo autor

Como pode-se observar na **Listagem 46**, a documentação do método `getFisicaByCpf` é inserida entre as Linhas 9 e 33. Na linha 9 é apresentado uma breve descrição do método/*endpoint* através da anotação `@Operation`. Em seguida, são documentados os possíveis códigos HTTP pela anotação `@ApiResponses`, que “abriga” três anotações `@ApiResponse`. A documentação para os códigos `400` e `404` (Linhas 22 a 31) são similares, sendo utilizadas informações de código (`responseCode`), descrição (`description`) e tipo de parâmetro utilizado (`content`). Note que o último parâmetro não foi especificado, apenas anotado com `@Content`. Já a documentação

para o código `200` (Linhas 51 a 60) é especificado tanto o tipo de dados a ser retornado (`mediaType = "application/json",`) quanto o esquema equivalente (Linha 57). A **Figura 22** ilustra a situação antes e após a inserção da documentação no método `getFisicaByCpf` definido pelo endpoint `/entidade/v1/cpf`. Os quadros e setas em vermelho indicam os pontos de mudanças com a inserção das anotações mencionadas.

Figura 22 – Comparativo antes e após a documentação do endpoint



Fonte: Elaborada pelo autor

4.5 Considerações finais

O conteúdo apresentado neste documento permite aos interessados iniciar a documentação de APIs REST, onde foi implementada as funcionalidades básicas e consideradas mínimas. Maiores detalhes podem ser obtidos na documentação do [springdoc-openapi](#) (SPRINGDOC-OPENAPI, 2021).

Referências

- BECHTOLD, S. *et al.* *JUnit 5 User Guide*. 2007. On-line. Versão 5.7.2. Disponível em: <>, Acessado: 27 de outubro de 2023. Citado 2 vezes nas páginas 6 e 8.
- ERL, T. *Service-Oriented Architecture: Concepts, Technology, and Design*. USA: Prentice Hall PTR, 2005. ISBN 0131858580. Citado na página 69.
- ERL, T. *SOA Principles of Service Design*. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2007. ISBN 0132344823. Citado na página 69.
- ERL, T. *et al.* *Web Service Contract Design and Versioning for SOA (Paperback)*. 1st. ed. USA: Prentice Hall Press, 2017. ISBN 0134767438. Citado na página 69.
- FOWLER, M. *et al.* *Patterns of Enterprise Application Architecture*. [S.l.]: Addison-Wesley Professional, 2002. Citado na página 57.
- GAMMA, E. *et al.* *Design Patterns: Elements of Reusable Object-Oriented Software*. [S.l.]: Addison-Wesley Professional, 1995. Citado na página 6.
- JUNIT. *JUnit framework*. 2022. [On-line], World Wide Web. Disponível em: <<https://junit.org/junit5/>>, Acessado em: 27 de outubro de 2023. Citado na página 7.
- ORACLE. *JavaBeans Spec*. 2021. [On-line], World Wide Web. Disponível em: <<https://www.oracle.com/java/technologies/javase/javabeans-spec.html>>, Acessado em: 27 de outubro de 2023. Citado na página 35.
- PRESSMAN, R. S. *Engenharia de software: uma abordagem profissional*, 7. ed. Porto Alegre: AMGH, 2011. Citado na página 6.
- SOMMERVILLE, I. *Engenharia de Software*, 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado na página 6.
- SPRING. *Spring Data*. 2021. [On-line], World Wide Web. Disponível em: <<https://spring.io/projects/spring-data>>, Acessado em: 27 de outubro de 2023. Citado na página 24.
- SPRING. *Spring Framework*. 2021. [On-line], World Wide Web. Disponível em: <<https://spring.io/projects/spring-framework>>, Acessado em: 27 de outubro de 2023. Citado na página 23.
- SPRINGAPI. *Spring Framework 5.3.9 API. This is the public API documentation for the Spring Framework*. 2021. [On-line], World Wide Web. Disponível em: <<https://docs.spring.io/spring-framework/docs/current/javadoc-api/index.html>>, Acessado em: 27 de outubro de 2023. Citado na página 75.
- SPRINGDOC-OPENAPI. *springdoc-openapi v1.5.9, Library for OpenAPI 3 with spring-boot By Badr NASS LAHSEN*. 2021. [On-line], World Wide Web. Disponível em: <<https://springdoc.org/>>, Acessado em: 27 de outubro de 2023. Citado 2 vezes nas páginas 70 e 79.

SWAGGER. *What Is the Difference Between Swagger and OpenAPI?* 2017. [Online], World Wide Web. Disponível em: <<https://swagger.io/blog/api-strategy/difference-between-swagger-and-openapi/>>, Acessado em: 27 de outubro de 2023. Citado na página 69.